Breaking the HBM wall: A Distributed Spatial Processing-Near-Memory Architecture using DUV ASICs and Deterministic Routing

Bowen Gu

October 20, 2026

Abstract

The memory wall has hardened into a physical wall: High-Bandwidth Memory capacity per package is bounded by interposer reticle limits[15], and the manufacturing economics behind HBM stacks—which consume roughly three times the wafer area per bit of commodity DRAM, at extreme power density—are structural rather than cyclical[25], just as the workload mix shifts toward capacity-bound jobs—including Mixture-of-Experts transformer inference, scientific stencil computation, climate modeling, seismic imaging, and large-scale numerical simulation—whose binding resource is resident memory, not peak FLOPs[3, 4, 26].

This paper proposes a distributed spatial Processing-Near-Memory (PNM) architecture that bypasses the HBM wall with commodity 192-bit LPDDR6 CAMM2 modules[16, 17] paired with mature-node DUV MAC ASICs, wired by a deterministic single-spine wormhole fabric of stateless Hardware Flit Repeaters and combinational lxy_repeater nodes that replaces runtime scheduling and cache coherency with coordinate arithmetic and closed-form latency bounds[13, 14]. A 512-node reference chassis provides 64 TB of attached memory (two 64 GB 192-bit modules per node, 384-bit, 512 GB/s node-local), 262 TB/s aggregate node-local bandwidth, and approximately 197 TFLOPS FP64 at approximately 15 W per node—8–10 kW per chassis against 0.25 MW for capacity-equivalent HBM—managed by one socketed router-bridge chip that a host—a Raspberry Pi or any third-party microcontroller—drives over a simple host link, handling topology discovery, workload dispatch, and the host's command stream[5].

The result is bit-exact-reproducible computation, deadlock freedom by construction, compile-time latency bounds, and a fraction of the thermal envelope, interposer dependency, and manufacturing complexity of monolithic GPU accelerators. Validation scope is stated explicitly: the machine-checked claims are byte-exact delivery and replay, doorbell refusal accounting, cycle-exact closed-form latency on a 100 MHz behavioral co-simulation, a real-checkpoint numeric forward pass that is greedy-token-identical to a BF16 reference, and a bit-exact integrated compute datapath—gating ASIC, fabric transport, and the selectable node MAC, plus a full decoder block—validated on real layer-0 weights (Sections 2.15, 2.20, 3.5, and 4.4); bandwidth, power, and nanosecond latencies are projections of characterized link and process assumptions, not silicon measurements.

Keywords: Processing-Near-Memory; Wormhole Switching; Deterministic Routing; Dimension-Order Routing; Spatial Computing; LPDDR6 CAMM2; DUV ASIC; Mixture-of-Experts Transformers; Stencil Computation; Roofline Analysis; Host Bridge; Raspberry Pi; High-Performance Computing

Introduction

1.1 The physical bottleneck

The prevailing paradigm for large-scale transformer inference relies on monolithic GPU accelerators tightly coupled to High-Bandwidth Memory (HBM) via advanced silicon interposers[2, 15]. This approach is hitting three hard physical limits that no process shrink or market cycle can resolve.[1] First, HBM capacity per package is bounded by interposer reticle limits: each HBM stack is a vertical assembly of DRAM dies connected through silicon vias, and the interposer that bridges these stacks to the GPU die has a fixed reticle-bounded area[15]. This caps the number of stacks that can be placed around a single GPU, which in turn caps the aggregate memory capacity per accelerator—currently 80 GB for the largest shipping parts. For capacity-bound workloads this ceiling is the binding constraint: the compute exists to process the data set, but the memory to hold it does not fit. Mixture-of-Experts (MoE) transformers are the most visible instance—hundreds of billions of parameters, most of them idle during any given forward pass[3, 4]—but the same profile recurs across sparse retrieval, scientific simulation state, and seismic image volumes.

Second, the manufacturing burden of HBM is concentrated and severe: HBM stacks consume roughly three times the wafer area per bit of commodity DRAM, and the 2025–26 DRAM shortage—driven by fabrication capacity shifting

from commodity DRAM toward HBM—demonstrates the supply fragility this creates[25].

Third, the power density of an advanced interposer stack is extreme: an H100-class GPU dissipates roughly 300 W in a package smaller than a paperback book, and the thermal management of stacked DRAM dies—which cannot tolerate junction temperatures above 85°C—limits the number of layers that can be reliably cooled. A single NVIDIA H100-class GPU with 80 GB of HBM can cost upwards of thirty thousand dollars—roughly $375 per gigabyte of attached memory—yet the majority of its transistor budget and thermal headroom is spent not on computation, but on moving data across hierarchical memory caches and network interfaces[2]. The architecture described in this paper sidesteps all three limits by replacing interposer-bound HBM with commodity LPDDR6 CAMM2 modules[16, 17]—each a standard JEDEC part with no interposer dependency, no advanced lithography requirement, and a thermal envelope of approximately 5 W—paired with a mature-node 14 nm or 28 nm DUV MAC ASIC. The Results section quantifies the alternative: a 512-node chassis provides 64 TB at approximately 15 W per node and approximately 8–10 kW per chassis, versus 0.25 MW and 819 GPUs for capacity-equivalent HBM.

1.2 Design philosophy: hardware as a static dataflow graph

This paper advances an alternative philosophy inspired by Unix pipelines[7], pure functional programming[6], operating-system microkernels[8], and the dataflow computing tradition[11, 12]. We treat the physical motherboard not as a passive substrate for a Von Neumann processor, but as an active, immutable dataflow graph. Each Hardware Flit Repeater (HFR) does exactly one thing—it forwards a flit to the next node—just as a Unix program does one thing well and composes via pipes. The hardware itself enforces functional immutability: flits are immutable once injected, HFRs are stateless functions of their input, and MAC ASICs produce deterministic outputs from static memory contents. The statelessness is a property of the data transport plane precisely: the fabric carries no global mutable state—no runtime page tables, no dynamic branch prediction, no cache coherency protocols, and no routing or arbitration state on the forwarding path. All mutable state is deliberately concentrated and accounted for: the pre-loaded routing bitmaps and configuration of every lxy_repeater and gate, the MoE expert map and gating weights held by the central router chip, the pending weight-upload command stream during POST, the KV cache banks and their eviction state, the dispatch records the firmware emits, and the POST/READY discovery state of the router's boot sequence. Each of these is (i) owned by exactly one mutable agent—the router chip or a node's controller—never by the forwarding path, (ii) initialized from a compile-time schedule, and (iii) replayable, so the deterministic-replay guarantee of Section 2.10 attaches to the stateful control plane exactly as it would to a stateless one. The entire machine is a single, spatially distributed pure function—a physical instantiation of a dataflow graph—making it a natural substrate for first-order, statically allocated functional programs (Section 2.14). The lxy_repeater extends the microkernel principle from software to silicon: it enforces a strict, non-bypassable physical separation between layers through a hardware bitmask comparison, with no configuration register to misprogram, no firmware to exploit, and no driver to crash[9]. This purity is not merely aesthetic: it is what makes bit-exact replay after faults (Section 4.1) and compile-time latency bounds (Section 4.3) possible at all.

1.3 Manufacturing realities: DUV versus EUV

A common objection to custom accelerator silicon is the perceived necessity of bleeding-edge process nodes. This architecture rejects that premise. Memory bandwidth, not transistor switching speed, is the binding constraint for inference and stencil workloads. The roofline analysis of Section 3.2 shows both flagship workload families sit deep in the bandwidth-bound regime at the reference machine balance[5], so a 14 nm MAC array never limits performance. A 14 nm or 28 nm Deep Ultraviolet (DUV) ASIC is therefore perfectly sufficient to saturate a local LPDDR6 bus[17], and mature nodes offer structural advantages that no market cycle can erase: dramatically higher yields than EUV lithography, established supply chains with predictable lead times, and the freedom to use large die areas for MAC arrays without the exponential cost curve of sub-7 nm fabrication. The 2025–26 DRAM shortage—driven by fabrication capacity shifting from commodity DRAM toward HBM, which consumes roughly three times the wafer area per bit[25]—further illustrates the fragility of an ecosystem built on interposer-bound, EUV-dependent memory. By pairing commodity LPDDR6 CAMM2 modules with mature-node logic, the architecture decouples memory capacity from the interposer and lithography bottlenecks that constrain HBM.

1.4 ASIC philosophy and historical precedents

General-purpose processors derive flexibility from massive instruction decoders, register files, branch predictors, and cache hierarchies. An ASIC burns transistors only on the logic that directly contributes to the target computation[10]. Our architecture applies this principle to mature DUV nodes: the DUV MAC ASIC contains no instruction decoder, no branch predictor, and no operating-system interface—only a fixed-function MAC array hard-wired to its local LPDDR6 bus. The design inherits the INMOS Transputer's philosophy of computing elements with direct, deterministic communication[12], but replaces its general-purpose processors with fixed-function MAC ASICs and its serial links with high-bandwidth LPDDR6 buses. The wormhole routing fabric draws on Dally and Seitz's work on deadlock-free routing:

deterministic routing is deadlock-free if and only if the channel dependency graph is acyclic[13, 14]. Our fabric guarantees acyclicity by construction through monotonically ordered virtual-channel classes (Section 4.4).

1.5 Related work

Processing-in-memory and near-memory computing have been studied for decades as remedies for the same bandwidth wall this paper attacks[1], and the recent survey literature organizes the design space around how far logic migrates toward the data and how that logic is programmed[35]. PIM-enabled instructions, for example, attach vector instructions to memory modules while preserving a conventional host ISA and a coherent memory view[36].

Three commercial lines define the current state of the art. UPMEM distributes simple programmable cores across the DRAM chips of standard DIMMs[31], later characterized in detail as a real PIM system[37]; Samsung's HBM-PIM embeds MAC units inside an HBM2 stack[32, 33]; and the AXDIMM family places an acceleration buffer on a host-attached DDR module[34]. All three share two properties: compute scales with the memory package it is welded to (an interposer-bound stack or a vendor-specific buffer), and the resulting memory remains part of a coherent host hierarchy, so the programming model inherits caches, coherence, and operating-system virtual memory.

The architecture in this paper departs from that line in three respects. First, the memory is unmodified commodity LPDDR6 CAMM2 (Section 2.4): socketed, field-replaceable, and interposer-free in the reference packaging, with the compute on a separate mature-node DUV ASIC beside the socket, so capacity scales by adding modules rather than by resizing stacks. Second, the modules are not part of a coherent host memory hierarchy: the chassis is a spatially scheduled dataflow machine in which the ahead-of-time compiler replaces coherence, caches, and runtime scheduling (Sections 1.2 and 2.7). Third, transport is a deterministic single-spine wormhole fabric with an end-to-end doorbell and closed-form latency (Sections 2.1, 2.9, and 3.4), so its correctness claims reduce to byte-exact delivery and replay—properties the co-simulation machine-checks (Section 2.15) rather than argues statistically. The routing theory that makes determinism deadlock-free is classical[13, 14]; the contribution here is the composition into a capacity-serving machine from commodity parts, together with the verification methodology that holds the composition honest.

The accelerator families that dominate current serving sit at the opposite end of the design space, and two of them bracket the capacity argument directly. Groq's static-dataflow language processing unit holds the entire model in on-chip SRAM (roughly 230 MB per chip), eliminating instruction fetch for sub-microsecond dispatch; Nvidia licensed the technology for $20 billion in late 2025 to anchor its LPX racks[38]. Cerebras' CS-3 waferscale engine scales the same SRAM-resident idea to 44 GB on a single wafer[39]. Both are bandwidth-dense but capacity-poor: on-chip SRAM costs one to two orders of magnitude more per bit than DRAM, so a 64 TB resident model is unreachable by construction (Section 4.5). Taalas' HC1 takes the opposite route and hard-codes the model: the weights are etched into a mask-ROM recall fabric on a reticle-sized 6 nm die, with SRAM only for the KV cache and adapters, reaching roughly 17,000 tokens per second per user on Llama-3.1-8B at roughly 200 W[40]—an order-of-magnitude validation of the capacity-bound decode thesis, at the price of freezing the model in silicon, since any model change is a two-metal-layer re-spin (Section 4.4). AMD acquired Taalas in August 2026, planning to pair its Instinct racks with the HC1-derived decode accelerators[41], the same prefill-heavy/compute-on-GPU and capacity-heavy/decode-offload split Section 4.5 argues for. PNM occupies the quadrant those systems leave open: commodity DRAM for the capacity the serving regime actually demands, a per-node DUV MAC for the arithmetic, and an ahead-of-time compiler that keeps the model software-reprogrammable, so the capacity economics of the MSIC are reached without its model lock-in.

1.6 Contributions

This paper makes five contributions. First, an architecture specification: the single-spine deterministic wormhole fabric with dimension-order on-board paths, the doorbell activation discipline, the four-class virtual-channel deadlock argument, and the reference 512-node, 64 TB chassis, with closed-form latency and spine-sizing rules (Sections 2.1–2.4, 2.9, 3.1, 3.4, and 4.3). Second, a five-stage ahead-of-time compilation pipeline that lowers HuggingFace safetensors models and typed source-language IRs onto physical coordinates, with capacity-aware expert placement, KV-cache budget reservation, and static dispatch schedules (Sections 2.6–2.8). Third, a machine-checking methodology for the transport claims: a deterministic cycle-exact co-simulation of the Verilog fabric whose five oracles verify byte conservation, zero misroutes, doorbell accounting against an independent software twin, kernel results against a golden model, and exact closed-form latency, including bit-exact replay, a discrete-event cross-check of the egress arbiter, a real-checkpoint numeric forward pass that is greedy-token-identical to a BF16 reference on the tested prompts, and a bit-exact integrated compute datapath—gating ASIC, fabric transport, and the selectable node MAC, plus a full decoder block—validated on real layer-0 weights (Sections 2.15, 2.20, and 3.5). Fourth, an explicit validation-scope discipline: every bandwidth, power, thermal, and cost figure is labeled as a projection, and every machine-checked claim is enumerated (Sections 3.5 and 4.4). Fifth, an open hardware and software artifact: the Verilog fabric, compute-unit library, and a CPU-less router bridge with a host link (SPI on a Raspberry Pi, or any third-party microcontroller), with matching Go and C firmware twins, the model compiler,

PCB and assembly models, and a reproducible test suite in the companion repository[28].

Methods

2.1 The Z-axis spine and repeater nodes

The network topology is a strictly deterministic single-spine tree—a spine-and-leaf fabric with exactly one spine. A central Z-axis spine runs vertically through the chassis, bridging parallel horizontal X/Y motherboards with high-density mezzanine connectors. Traffic between nodes on the same layer follows exactly one dimension-ordered X/Y path; traffic between layers additionally enters the spine at its own layer's attachment, travels monotonically along the spine to the destination layer's attachment, and continues across the destination board's dimension-ordered path. Because there is exactly one legal route between any pair of nodes, routing requires no search algorithm, no arbitration, and no adaptive logic—only coordinate arithmetic.

At each layer, an lxy_repeater evaluates the incoming flit header against a local hardware ID bitmask. Implemented as a combinational tree of XOR and AND gates on a 14 nm DUV node, this comparator operates in approximately 50–100 picoseconds without clocks, queues, or software intervention in the compare path. The comparator's match decision is registered on the local Network-on-Board (NoB) clock edge; clock-domain crossings use standard source-synchronous forwarding between HFRs. Matching flits are gated onto the local X/Y NoB; non-matching flits proceed along the spine. The lxy_repeater boundary acts as a rigid physical microkernel barrier[9]. Direction is decided ahead of time, never at runtime: the AOT compiler (Section 2.8) emits a deterministic, immutable routing table for every repeater, loaded into the repeaters beside each ASIC node during initialization; each lxy_repeater steers flits solely from its pre-loaded table, with nothing to adapt, arbitrate, or reconfigure on the data path.

The spine is the fabric's only bisection, and it is sized by arithmetic, not by assertion. Let α be the cross-layer traffic fraction of the dominant workload—a compile-time constant derived by the AOT mapper from the resident placement of experts or stencil subdomains (Section 2.8). If the dominant cross-layer trip moves S bytes (a kilobyte-class token plus its returned hidden state, $S \approx 8$ KB), then a spine provisioned at B bytes per second per direction sustains at most $R = B/(\alpha S)$ cross-layer dispatches per second. The reference chassis provisions the spine as ≈128 shared parallel 16 GB/s lanes (≈2 TB/s aggregate per direction, shared by all 512 nodes), which sustains $\approx 3\times10^8$ cross-layer dispatches/s under naive random expert placement ($\alpha = 7/8$) and rises past 10^9 dispatches/s once hot-expert replication and layer-affine placement cut α to ≈1/4 (Section 2.12).

Each generated token costs 270 fabric dispatches—30 dense attention projections plus 240 MoE expert activations for the

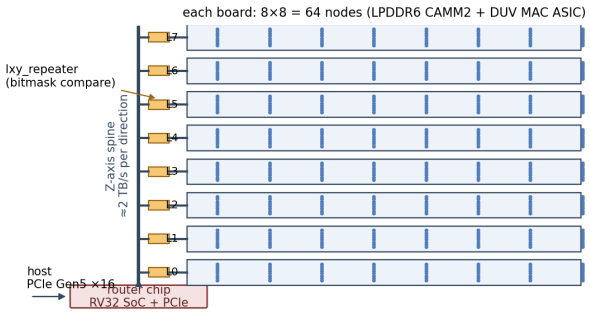


Figure 1: The reference chassis: eight X/Y node boards of 64 nodes each, stacked on a single Z-axis spine. The router chip at the spine root owns POST discovery, routing tables, weight upload, and token dispatch; every other data-path element is a stateless repeater or a fixed-function node.

Gemma-4-class reference workload (Section 2.6)—so the honest end-to-end token rates are $\approx 1.1\times10^6$ tokens/s at $\alpha = 7/8$ and $\approx 3.7\times10^6$ tokens/s at $\alpha \approx 1/4$: the dispatches/s figures are fabric throughput, and the token rate follows by dividing by the per-token dispatch count. The rule is exact because every route is known to the compiler. Intra-layer stencil halo traffic never enters the spine (Section 2.2). A fat-spine variant—adding lanes toward the root—scales this bound for larger stack heights at the cost of connector pins, which are the dominant spine cost.

The system therefore reduces to four kinds of hardware (Figure 1). Commodity DRAM (192-bit LPDDR6 CAMM2) holds all weights and state. A DUV MAC ASIC beside each module performs the only computation. Stateless Hardware Flit Repeaters (HFRs) and the combinational lxy_repeaters form the entire data path. And one central router bridge—the only stateful silicon in the machine—owns everything that is not streaming data: it performs Power-On Self-Test and topology discovery (Section 2.5), accepts the host's command and weight stream over a simple host link (Section 4.2), evaluates the coordinate mapping for dynamic MoE expert routing (Section 2.8), and injects every JIT-dispatched token into the fabric. The bridge has no processor of its own: the host—a Raspberry Pi or any third-party microcontroller—runs the boot sequence and the dispatch loop, so the same firmware serves a Pi, an MCU, or a workstation. Everything on the hot path is pure transport; everything with state lives in a single chip at the spine root.

2.2 The intra-layer fabric: dimension-order routing

Within a layer, HFRs form a direct X/Y grid that mirrors the motherboard's physical socket layout. On-board routes use X-then-Y dimension order: a flit travels along the X axis to the destination column, then along the Y axis to the destination socket[13, 14]. Dimension-order routes are deterministic,

minimal, and free of cyclic channel dependencies by the standard results. A halo exchange between neighboring grid cells is a single-hop route between adjacent sockets that never touches a lxy_repeater or the spine.

Every fabric link—on-board and spine alike—is characterized as a 128-bit-wide source-synchronous channel at 1 GHz, providing ≈16 GB/s per link. The HDL sketch and the co-simulation harness model the byte-oriented flit stream at one byte per cycle on a 100 MHz testbench clock; the 1 GHz 128-bit physical link is never clocked in simulation. The byte-serial 100 MHz model is therefore slower than the characterized link by roughly 160×—a 10× clock gap times sixteen bytes per cycle—understating per-link bandwidth by that factor while preserving the routing-decision logic that is the point of the model. The nanosecond latencies of Section 3.4 are projections of the closed-form hop-count expression at the characterized 16 GB/s link rate, not measurements of the 100 MHz testbenches; the testbenches verify routing, framing, and the doorbell discipline cycle-exactly at their own clock, and Table 3 reports those cycle counts as the verification record. Cross-node traffic is bounded by these 16 GB/s links, which is deliberately separate from the ≈512 GB/s each node can read from its own memory (two 192-bit LPDDR6 modules): the aggregate figure in Section 3.1 is a sum of node-local bandwidths, and any workload that must move data between nodes is fabric-bound at the link rate.

The fabric provides four routing patterns, selected by the AOT compiler from the algorithm's communication topology rather than programmed at runtime:

Dimension-order X→Y (intra-layer). Nearest-neighbor stencils, halo exchanges, and any communication confined to a single board. The route never enters the spine, so its latency is a compile-time constant independent of chassis depth: a single dimension-order hop for a nearest-neighbor halo exchange, plus PE pipe delay. This is the cheapest pattern and should be preferred whenever data locality permits.

Spine descent (cross-layer unicast). Weight-stationary linear algebra (matvec), MoE expert dispatch, and any pattern where the injector reaches nodes on deeper layers. Each hop adds exactly one spine stage; the closed-form latency of Section 3.4 accounts for this deterministically.

Reverse-path merge (egress collection). Reduction trees, result aggregation, and any many-to-one pattern. The forward path is a demux tree making no arbitration decisions; the reverse path is a pure merge tree whose round-robin grants are held for whole packets (never torn) and whose dependency graph is acyclic, so deadlock-free delivery is guaranteed structurally (Section 4.3).

Spine descent + fan-out (broadcast/multicast). Weight upload, parameter distribution, and any one-to-all or one-to-many pattern. The source node's uplink is the bottleneck; the compiler schedules broadcast bursts during idle phases (POST boot) or overlaps them with compute to hide serialization.

The compiler selects among these patterns during its route-planning stage: it computes each message's source-destination pair, checks whether the pair shares a layer (dimension-order suffices), and otherwise inserts spine hops into the source-routed header. No runtime routing decisions exist in the fabric itself—every flit carries its complete route from the moment of injection.

2.3 The motherboard and isothermal manifold

The horizontal plane consists of rigid X/Y motherboards populated with paired LPDDR6 CAMM2 memory sockets and DUV MAC ASIC sockets (Section 2.4). Stacked vertically, these boards create a severe thermal challenge: LPDDR6 modules throttle above 85°C. At the reference operating point, each node dissipates approximately 15 W (about 5 W DRAM, 9 W MAC array, 1 W fabric share), a fully populated 64-node board about 1 kW, and an eight-layer chassis about 8–10 kW including fabric and control losses. The chassis incorporates an Isothermal Parallel Manifold—rigid vertical fluid channels that inject identical-temperature coolant (30°C) simultaneously across all stacked layers. Balance is enforced, not assumed: each branch is fitted with a calibrated flow orifice sized during chassis assembly, holding each branch's coolant temperature rise within a ±2°C band. At approximately 15 L/min per chassis (water, 10°C coolant temperature rise), the worst-case DRAM junction temperature remains at or below 70°C—15°C below the throttling threshold.

2.4 CAMM2 integration, packaging, and sideband signaling

Each compute node centers on JEDEC LPDDR6 CAMM2 memory modules (the socketable form factor that succeeds JESD318's LPCAMM2)[16, 17]. LPDDR6 (JESD209-6) organizes its bus as 24-bit channels, each made of two 12-bit sub-channels; one CAMM2 module is 8×24 bits (four 48-bit channel groups), i.e. a **192-bit** interface. The reference configuration pairs **two 64 GB modules per node** at 10.7 Gb/s/pin, so the node presents a **384-bit** (dual-module) bus yielding ≈512 GB/s, while capacity stays 128 GB per node (64 TB per 512-node chassis) and each module remains an unmodified commodity part. The DUV MAC ASIC sits adjacent to—rather than inside—the DRAM die, making this Processing-Near-Memory (PNM) rather than true Processing-in-Memory (PIM). A JEDEC LPDDR6 interface requires a host-side memory controller, and this design integrates that controller into the DUV MAC ASIC itself: the controller lives on the motherboard in the ASIC socket beside each CAMM2 module, its LPDDR6 PHY pins of Table 1 being the memory side of the interface (Section 2.16), with no separate controller chip on the board. Two packaging variants define adjacency:

Variant A — Socketed ASIC on the motherboard (reference design). Each node's two CAMM2 sockets flank one adjacent low-profile ASIC socket (land-grid-array or card-edge

mezzanine), whose controller drives both 192-bit PHYs. The ASIC is socketed rather than soldered: nodes can be populated, depopulated, and replaced in the field, matching the POST discovery sequence (Section 2.5) and the failure model (Section 4.1). The memory modules remain unmodified commodity parts. Placing each socket within roughly 20 mm of the ASIC keeps both LPDDR6 interfaces short enough for commodity-grade signal integrity.

Variant B — Module-integrated ASIC (high-density option). The DUV MAC ASIC is mounted directly on the CAMM module substrate beside the DRAM packages. This yields the shortest memory interface and lowest interface power, at three costs: the module becomes a custom part, ASIC heat sits adjacent to the DRAM, and a failed ASIC retires the entire module.

Direct BGA attachment of the ASIC to the motherboard was considered and rejected: a permanent solder joint defeats field serviceability and contradicts the reconfigurable-inventory philosophy of Section 2.5. The reference configuration in the Results section assumes Variant A.

Because the CAMM2 connector pins are fully occupied by the memory bus, four sideband signals travel on a separate low-pin-count board-level sideband header (in Variant B, a short flex tail from the module substrate). Each is a single wire from the node, and each runs as part of a daisy-chained control net that climbs the spine to the central router chip; the net is a low-speed control plane entirely separate from the data-path NoB:

DOORBELL_TRIG: asserted by the node's DMA engine only when (a) the received byte count equals the message-length field in the flit header, (b) the end-to-end message CRC validates, and (c) the CRC-protected destination field equals the node's own coordinate. Because HFR forwarding along a single path is strictly in-order, byte-count equality detects complete delivery; a stream aborted mid-message can never satisfy all three conditions. The full state machine is specified in Section 2.9.

DOORBELL_ACK: asserted by the same DMA engine in the same cycle as DOORBELL_TRIG, and forwarded up the sideband net. It is the source's completion signal, and its single-wire cost is the entire acknowledgment overhead of the design: there is no ACK packet, no sequence number, and no extra latency on the data path. A source that has not seen DOORBELL_ACK within its compile-time latency budget infers a lost or un-executable message and raises NODE_ERR for that destination (Section 4.1). The ACK wire is the loss detector that turns the doorbell's refusal-to-fire into a recoverable, observable event.

NODE_ERR: a fatal fault line asserted on the sideband control net, streaming directly to the central router chip (Section 4.1). It does not travel on the data path, so it remains usable when the node's NoB or DMA path is dead.

TOPOLOGY_RDY: asserted during Power-On Self-Test once voltage margins and link training succeed.

Table 1: Board-level pin and trace outline (per node; first-order estimates).

Interface	Pins	Routing
CAMM2 sockets (LPDDR6, 192-bit, $\times$2/node)	$\approx$600–650 each	192 DQ, 24 DQS pairs, CA/CS/CK, power/ground; each $\leq$20 mm to the shared ASIC socket
ASIC socket (Variant A)	$\approx$800–850	LPDDR6 PHY $\approx$290, fabric 128-bit $\times$2 $\approx$270, sideband 4, clock/test $\approx$20, power/ground remainder
Sideband header	6	4 signal + ground (+3.3 V); daisy-chained control net up the spine
Spine mezzanine	128 $\times$ lanes + control	per-link 128-bit source-synchronous with forwarded clock

Board outline and pin budgets. The motherboard's wiring is entirely point-to-point and length-deterministic—the property that makes the closed-form latencies of Section 3.4 possible. Table 1 itemizes the four connector populations a node touches; the counts are pin-budget outlines, not JEDEC exactness, since the LPDDR6 CAMM2 class is forward-looking while the current LPCAMM2 class (JESD318) is a 644-pin module[16]. The ASIC side of the same budget, with full electrical specifications, is tabulated in Section 3.6.

Every trace is impedance-controlled and length-matched within its group. The LPDDR6 escapes from the two CAMM2 sockets to the shared ASIC socket run $\leq$20 mm each on dedicated DDR routing layers as tightly matched bundles—the shortest legal memory route on the board. The fabric links are point-to-point HFR-to-HFR runs on their own layers, driven source-synchronously: the clock is forwarded beside the data, so per-link skew is absorbed by the receive FIFO and the wire delay is the single deterministic term the compiler's closed form uses. A 64-node board is estimated at 10–12 routing layers: two DDR escape planes, two fabric planes, a spine-breakout region, and power/ground planes between them. The four sideband signals share one low-speed daisy-chain trace pair climbing the spine to the central router chip; their single-wire-per-signal cost is the entire control-plane wiring budget of the design.

2.5 Initialization and the POST discovery sequence

The firmware state machine boots the chassis through five deterministic phases, each completing before the next begins. The host's firmware drives the sequence; no node participates until the phase that addresses it. The two phases that program the machine – the routing table and the gating state – are machine-checked against each other in the firmware's two implementations (Section 3.5). The state machine is:

RESET $\rightarrow$ POST_DISCOVERY $\rightarrow$ ROUTING_TABLE $\rightarrow$ WEIGHT_UPLOAD $\rightarrow$ MOE_LOAD $\rightarrow$ READY

Phase 1: POST_DISCOVERY. The router emits a ping frame; each layer's lxy_repeater propagates it along the local X/Y NoB. Every populated node that completes link training and Built-In Self-Test asserts TOPOLOGY_RDY. The router aggregates responses into a live inventory: for each coordinate (L, X, Y), the inventory records the 8-bit Module ID ($X[3:0] \parallel Y[3:0]$), the per-node bandwidth (≈512 GB/s from two 192-bit LPDDR6 CAMM2 modules), and the node's status (ready or down). At the reference scale, 512 LPDDR6-class links train in milliseconds each and run in parallel; the ping/inventory sweep completes in microseconds, and the router aggregates the inventory in well under one second before any weight is loaded. The inventory is the Instruction Set Architecture for this boot instance: it defines the set of live coordinates, the physical topology, and the deterministic latency between every pair.

Phase 2: ROUTING_TABLE. The router programs every lxy_repeater and HFR in the chassis with its routing-table entry, using the 11-bit routing bitmap format of Section 2.7 (Stage 4). The router computes these bitmaps from the inventory: for a node at (L, X, Y), the lxy_repeater at layer L receives a bitmap matching Layer L; the node itself receives a bitmap matching its full coordinate with AXIS = Y and DIST = Y. The bitmaps are loaded via the sideband control net during this phase; the fabric's deterministic single-path structure means every bitmap is a function of the two coordinate pairs, not a search result. In the co-simulation harness, the bitmaps are embedded in the generated Verilog topology (Section 2.15), so this phase is a no-op; in production, the sideband daisy chain carries the bitmaps from the router to each repeater and node.

Phase 3: WEIGHT_UPLOAD. The router streams weight tensors to every node's LPDDR6 bank. For each node, the router constructs a weight-upload command: a flit containing the target Layer ID, Module ID, payload length, and the weight bytes (synthesized from the model compiler's output, Section 2.7). The wire format is:

CMD (0x01) — LAYER — MODULE_ID — LEN_LO — LEN_HI — payload... — CRC_HI — CRC_LO

The payload carries the weight data for the node's assigned compute unit: BF16 FMA weights for MoE experts, BF16 systolic array weights for attention projections, FP32 constants for layer norms, and INT8 weights for quantized inference. Each upload command also records the compute unit type (CUType), the data type (DType), the model layer index, and the tensor role (expert, attention, layernorm, embedding) so the host-side driver can verify weight upload integrity per unit type. The router sorts commands by (L, X, Y) for deterministic ordering and streams them in lexicographic coordinate order, ensuring that nodes on the same layer receive their weights before the next layer's upload begins. A fully populated 64 TB chassis loads in approximately 12–15 minutes at a realistic sustained fraction of the dual PCIe Gen5 x16 uplinks (Section 4.2); at the ≈128 GB/s unidirectional aggregate across the two uplinks, the line-rate bound is roughly 8.5 minutes, so the 12–15 minute figure reflects the achievable sustained fraction rather than the theoretical ceiling. The reference design also supports per-node boot flash on the sideband header (roughly two minutes through 512 parallel streams).

Phase 4: MOE_LOAD. The router loads the MoE gating state into its own SRAM. Expert selection lives on the per-node gating ASIC (Section 2.17.1), which evaluates it at runtime to determine which expert handles each token. The RTL's moe_gating unit (Section 2.17) implements the learned-projection datapath—logits = hidden × router_weights followed by top-k selection—with projector weights streamed over the host link into the per-node gating ASIC's weight SRAM. Because this work ships no end-to-end-trained projector, the dispatch path the co-simulation actually exercises routes each token through a deterministic token-seeded routing hash evaluated in router firmware (Section 2.12), which produces every routing decision quoted in Section 3.5; the RTL projection datapath and the hash share the top-k selection shape but not the learned behavior or expert identity. A learned 128-expert BF16 projector at the reference model's gating dimension needs roughly 0.7 MB of weights (128 × 2,816 × 2 B)—exactly the weight store the shipped gating ASIC provides—so only its per-token evaluation latency is a projection (Section 4.4). In the co-simulation, this phase records the gating state as loaded; in production, the weights arrive over PCIe from the host before execution begins.

Phase 5: READY. The chassis is fully initialized. The host exports a topology schema describing physical coordinates, per-link bandwidth, and deterministic latency between each pair—the closed-form latency is hop count × per-hop delay plus message serialization at the link rate (Section 3.4), not a search result. The AOT compiler (Section 2.8) then lowers the user IR against this exact schema, so the ISA for this boot instance exists within a second of power-on. The host enters the dispatch loop: for MoE workloads, it dispatches tokens to the experts the gating ASICs selected; for stencil workloads, it triggers halo exchanges at the scheduled timestamps. The dispatch loop processes each token through every model layer in sequence: for each layer, the firmware first dispatches to the dense attention node (round-robin by model layer), then checks whether any KV cache bank on that layer exceeds the offload threshold—if so, it dispatches a KV offload command and triggers a reload cycle—and finally dispatches to the top-k MoE expert nodes identified by the gating network. Each dispatch constructs a wormhole flit carrying the [Layer ID — Module ID] header, a CTRL byte encoding the virtual-channel class, the token payload, and the end-to-end CRC, and injects it into the fabric. The dispatch record tracks the layer, phase (dense, moe, or kv_offload), target node, expert index, flit byte count, KV action, compute unit type, and tensor role for host-side verification.

The entire boot sequence is deterministic and bounded. Every phase completes in known time: discovery in $<$1 s, routing-table load in $<$1 s (512 node bitmaps at 6 bytes each via

sideband), weight upload in approximately 12–15 min (64 TB through the dual PCIe Gen5 x16 uplinks at a realistic sustained fraction of the unidirectional rate), MoE load in <1 ms. The chassis does not enter the READY state until every phase completes successfully; if any phase fails (link training timeout, CRC mismatch on a weight upload, routing-table load NAK), the router raises NODE_ERR for the affected coordinate and continues in degraded mode (Section 4.1).

2.6 Software stack and execution model

The software stack is a narrow, layered pipeline that compiles high-level source languages onto the physical chassis. Unlike conventional GPU software stacks—which involve runtime drivers, kernel-mode scheduling, memory management units, and dynamic allocation—the PNM stack is entirely ahead-of-time: every weight, every route, and every dispatch timestamp is fixed before execution begins. The stack has four layers.

Layer 1: Source-language front ends. The architecture supports three source-language compilers that emit a typed intermediate representation (IR)[18] targeting the chassis's compute units. The R compiler (CompileR) parses a subset of R—assignments, arithmetic, aggregations (sum, mean, min, max), comparisons, and function calls—and emits FP64 IR instructions targeting the ‘fp64_fma‘ unit. The Haskell compiler (CompileHaskell) parses function definitions, guards, pattern matching, and higher-order aggregations (sum, product, minimum, maximum) and emits the same FP64 IR, supporting recursive function inlining and user-defined function composition. The HLSL compiler (CompileHLSL) parses GPU shading language—float declarations, arithmetic, intrinsic calls (dot, lerp, clamp, saturate, smoothstep, step, rcp, sqrt), and vector constructors—and emits FP32 ALU IR instructions targeting the ‘fp32_alu‘ unit. All three compilers are deterministic, side-effect-free, and produce IR that the downstream pipeline consumes without modification.

Layer 2: The intermediate representation. The IR is a typed, register-based instruction set with two precision domains. The FP64 domain (R and Haskell) provides: f64.load, f64.store, f64.add, f64.sub, f64.mul, f64.fma, f64.neg, f64.div, f64.min, f64.max, f64.cmp, f64.mov, and f64.const. The FP32 ALU domain (HLSL) provides: alu.load, alu.store, alu.const, alu.add, alu.sub, alu.mul, alu.div, alu.min, alu.max, alu.cmp, alu.dot, alu.lerp, alu.clamp, alu.rcp, and alu.mov. The IR is deliberately minimal: no loops, no branches, no dynamic memory allocation. This constraint is what makes the downstream AOT pipeline sound: every instruction maps to a fixed-function hardware unit, and the compiler can reason about the entire execution trace at compile time. Complex control flow—loops, conditionals, recursion—is lowered by the source-language front end into straight-line IR with explicit data dependencies, which the hardware executes as a static dataflow graph.

Layer 3: The AOT compilation pipeline (Section 2.7). The five-stage pipeline—ingestion, partitioning, kernel mapping, coordinate assignment, and dispatch scheduling—takes the IR and the model weights and produces a chassis schema and a ‘.pnm‘ program. The compiler assigns each IR instruction to a specific node's compute unit, routes the data dependencies through the fabric, and schedules every message injection to respect buffer credits and link bandwidth. The output is a static, immutable execution plan that the hardware follows without runtime indirection.

Layer 4: The firmware and runtime. The host's firmware (Section 2.5) boots the chassis over the host link, loads weights, and runs the READY-phase dispatch loop, managing the KV cache (Section 2.12) with FIFO eviction of the oldest entries when capacity is exhausted. A C port targets MCU implementations (ARM Cortex-M/R, RISC-V) for embedded deployments, using static allocation only—no ‘malloc‘, no dynamic memory, no operating-system dependency. The C port mirrors the Go firmware's data structures: a ‘firmware_t‘ struct holds the complete state (routing table, MoE map, weight commands, KV cache, dispatch records) as statically sized arrays, with configuration constants that scale by compile target—under ‘PNM_MCU‘ reduced limits for small microcontrollers (up to ‘PNM_MAX_LAYERS=2‘, ‘PNM_MAX_NODES_TOTAL=32‘, ‘PNM_MAX_EXPERTS=16‘, and ‘PNM_KV_CACHE_DEPTH=64‘), and the full SoC defaults otherwise (‘PNM_MAX_LAYERS=8‘, ‘PNM_MAX_NODES_TOTAL=512‘, ‘PNM_MAX_EXPERTS=256‘, and ‘PNM_KV_CACHE_DEPTH=4096‘); it implements the identical five-phase boot and dispatch loop and compiles cleanly with ‘gcc -Wall -Wextra -std=c11‘ against the Go firmware's golden model.

The execution model is weight-stationary: every parameter and activation buffer resides in a fixed LPDDR6 bank at a known coordinate, and computation is triggered by data arrival (the doorbell of Section 2.9), not by instruction fetch. The MAC array reads weights from the local banks at the full 512 GB/s bus rate, executes the resident kernel, and returns the result through the fabric. There are no runtime page faults, no TLB misses, no cache coherence invalidations, and no operating-system context switches. The hardware is a physical dataflow graph; the software is the static assignment of that graph to silicon.

2.7 The compilation pipeline

The compiler translates a model definition and its pre-trained weight tensors into a static dataflow graph that maps directly onto the physical chassis. The primary input is a safetensors archive—a flat binary of weight tensors with a JSON header indexing each tensor by name, dtype, shape, and byte offset—paired with a model configuration that specifies the computation graph: layer count, hidden dimensions, attention heads, expert counts, and activation functions. The output is a concrete assignment of weights, kernels, and routing tables

to specific physical coordinates, plus a dispatch schedule that the central router chip executes at runtime. The backend does not target CUDA, SIMD, or a conventional instruction set. It targets the physical topography schema exported during POST (Section 2.5).

This is a fundamentally different compilation model than conventional GPU software. A CUDA compiler generates an instruction stream that a general-purpose core fetches, decodes, and executes against a shared memory hierarchy. The PNM compiler generates neither instructions nor a program counter: the output is a spatial assignment of weights and kernels to physical addresses, where the address IS the coordinate and the doorbell IS the program counter. Computation is triggered by data arrival, not by instruction fetch. The compiler's job is to make the static dataflow graph correct by construction, so that the hardware can execute it with zero runtime overhead.

The pipeline proceeds in five stages, each deterministic and side-effect-free (Figure 2).

Stage 1: Ingestion. The compiler reads two files from a HuggingFace model directory: the safetensors index (‘model.safetensors.index.json‘) and the model configuration (‘config.json‘). The index maps each named tensor to its shard filename; the config specifies the computation graph—layer count, hidden dimensions, attention heads, expert counts, and activation functions. Crucially, the safetensors index does not store tensor shapes: the compiler infers shapes from tensor names using naming conventions and the config parameters.

For example, ‘model.layers.0.self attn.q proj.weight‘ maps to $[\text{HiddenSize}, \text{HiddenSize}]$; ‘model.layers.0.experts.3.gate_up_proj‘ maps to $[2 \times \text{MoEIntermediateSize}, \text{HiddenSize}]$; ‘model.layers.0.experts.3.down_proj‘ maps to $[\text{HiddenSize}, \text{MoEIntermediateSize}]$; ‘model.embed_tokens.weight‘ maps to $[\text{VocabSize}, \text{HiddenSize}]$; and ‘model.layers.0.router.proj.weight‘ maps to $[\text{NumExperts}, \text{HiddenSize}]$. Each tensor's byte length is computed as $\text{params} \times 2$ for BF16 (2 bytes per element). The catalog records the name, shard, byte length, element count, and data type for every tensor. For a LLaMA-class MoE model, this yields on the order of 200–300 named tensors per layer (attention projections, feed-forward gates, layer norms, routing weights) plus shared tensors (embeddings, output head, global norms). The safetensors format is weight-only: it stores no computation graph, no gradient tape, and no optimizer state, which is exactly what inference compilation needs. The compiler reconstructs the graph from the config and the tensor naming conventions, not from the archive itself.

Stage 2: Capacity-aware partitioning. Each reference node holds 128 GB of LPDDR6 (Section 3.1). The partitioner assigns tensor shards to physical nodes subject to two constraints: (a) no node exceeds its 128 GB memory budget, and (b) tensors that participate in the same kernel invocation must be co-located or connected by a known route. The partitioning proceeds in three sub-phases.

Sub-phase 2a: Layer-to-physical-layer mapping. The compiler divides the model's hidden layers evenly across the chassis's physical layers: $\text{model_layers_per_physical} = \lceil \text{NumHiddenLayers}/\text{ChassisLayers} \rceil$. For a 30-layer model on a 4-layer chassis, each physical layer owns 8 model layers (the last owns 6). Physical layer pl owns model layers $[pl \times 8, (pl+1) \times 8)$.

Sub-phase 2b: Expert distribution. Within each physical layer, the compiler distributes experts round-robin across the layer's $B_x \times B_y$ nodes. For model layer ml and expert index $expIdx$, the target node is $nodeIdx = expIdx \bmod (B_x \times B_y)$, with coordinates $x = \lfloor nodeIdx/B_y \rfloor$, $y = nodeIdx \bmod B_y$. Each expert contributes two tensors: ‘gate_up_proj‘ of size $[2 \times \text{MoEIntermediateSize}, \text{HiddenSize}]$ and ‘down_proj‘ of size $[\text{HiddenSize}, \text{MoEIntermediateSize}]$, both $\times 2$ bytes for BF16. A 70B-class MoE model's full expert population occupies on the order of 70–80 GB of BF16 weight—roughly 0.6 GB per expert at a typical 128-expert count—so the population spans the expert-parallel node pool rather than fitting on a single node, and the per-node post-weight budget (Section 2.7, sub-phase 2d) decides how many resident experts a node can host alongside its KV share. Hot experts—identified from offline activation histograms—are replicated across k nodes, placed on the same layer to avoid cross-layer spine traffic for the most frequently dispatched tokens (Section 2.12).

Sub-phase 2c: Dense components. Attention projections (Q, K, V, O), the dense MLP (gate, up, down projections), and two layer norms are assigned to a dedicated attention node, round-robin by model layer: $attnNodeIdx = ml \bmod (B_x \times B_y)$. This places all dense operations for a given model layer on a single node, eliminating cross-node traffic for the attention path. Embeddings are sharded across all layer-0 nodes by token-ID range: $\text{rowsPerNode} = \lfloor \text{VocabSize}/\text{nodesLayer0} \rfloor$, with the remainder distributed to the first nodes. The final layer norm goes to a rotating layer-0 node: $normNodeIdx = \text{NumHiddenLayers} \bmod \text{nodesLayer0}$. Router gating weights ($router.proj.weight$) are assigned to a sentinel node at coordinate $(-1, -1, -1)$—the router chip itself, not a compute node.

Sub-phase 2d: KV cache reservation. On every node that holds attention Q tensors, the compiler reserves space for the KV cache: $\text{kvEntryBytes} = \text{HiddenSize} \times 4$ (K and V, each 2 bytes in BF16, per hidden dimension)—the reference co-simulation and RTL kv_cache_bank use a fixed 512-byte frame per entry, which bounds the implemented capacity below the model formula for large hidden sizes. The KV budget per node is 80% of the post-weight budget—the 128 GB capacity minus the node's resident weight bytes—capped at 16,384 entries (a 16K context window). If the budget is exhausted, the firmware evicts the oldest entries (Section 2.12).

For models whose individual tensors exceed node capacity—extreme tensor-parallel scenarios—the partitioner shards the tensor across multiple nodes on the same layer, and the compiler inserts a local reduction kernel that merges partial results via single-hop on-board routes that never enter the spine. This is a special case: most production MoE models fit comfortably within the per-node capacity at FP16 or INT8, which is the architectural sweet spot the memory sizing was designed for (Section 3.3).

Stage 3: Kernel mapping. Each tensor operation in the graph is mapped to a hardware kernel and a compute unit type. The mapping is role-based: MoE expert weight-streaming uses BF16 FMA (weight-stationary multiply), attention QKV projections use BF16 systolic MAC arrays, layer norm uses FP32 ALU (for the division in variance computation), and the gating network's softmax uses FP32 ALU MIN/MAX/CMP operations. The reference kernel set includes: dot (matrix-vector multiply, for linear projections), accum (running accumulation, for softmax denominators and layer norm variance), sum (element-wise addition, for bias and residual connections), and echo (pass-through, for validation nodes and pipeline staging). The DUV MAC ASIC's fixed-function array executes the kernel on data present in its local LPDDR6 bank. The compiler emits the kernel name, resident weights (as hex bytes in the .pnm directive format), and bias constant for each assigned node. For MoE routing weights, the central router chip holds the gating network and evaluates the closed-form coordinate function at runtime (Section 2.8); the per-expert kernels are static and pre-loaded.

The key insight is that the kernel set is deliberately small and fixed. There is no general-purpose instruction decoder, no branch predictor, and no speculative execution on the node. Each kernel is a pure function of its input payload and resident weights, which is what makes the doorbell mechanism (Section 2.9) sound: the hardware can assert DOORBELL_TRIG with confidence that the node will produce a deterministic result, because there is no possibility of a page fault, a cache miss, or a branch misprediction changing the outcome. The compiler's kernel mapping is a many-to-one function—many tensor operations map to the same dot kernel, differing only in their weight dimensions—and the hardware executes them all identically.

Stage 4: Coordinate assignment and route computation. The partitioner's node assignments are translated into physical [Layer ID — Module ID] coordinates. The compiler then computes two static lookup tables.

The routing bitmap table: for each node at (L, X, Y), the compiler computes an 11-bit bitmap: bits [10:7] hold the 1-based Layer ID ($(L + 1) \ll 7$), bit [6] selects AXIS (0 = X), bit [5] selects SIGN (0 = +), and bits [4:0] hold the Y-distance from the lxy_repeater. The lxy_repeater at layer L receives a bitmap matching Layer L; the node itself receives a bitmap matching its full coordinate with AXIS = Y and DIST = Y. These bitmaps are loaded into the fabric during POST (Section 2.5) and are immutable within a boot session.

The MoE expert map: for every tensor with role "expert_gate_up," the compiler records a mapping from (model_layer, expert_idx) to the physical NodeID (L, X, Y) that holds that expert's weights. At runtime, the router chip evaluates the gating network, selects the top-k experts, and looks up this table to determine the destination coordinates. The table is static and loaded into the router's SRAM during Phase 4 of POST.

Because the fabric is a single-spine tree with dimension-order on-board routing, there is exactly one legal route between any pair of coordinates, and the route is an $O(1)$ arithmetic function of the two coordinate pairs (Section 2.8). No pathfinding algorithm is invoked. The compiler also computes the deterministic latency for every route: hop count times per-hop delay plus message serialization at the link rate (Section 3.4). These latencies become the compile-time budgets that sources use to detect failures (Section 4.1). The complete routing table is a static, immutable data structure loaded into the chassis during POST (Section 2.5).

Stage 5: Dispatch schedule generation. The AOT mapper produces a static execution schedule that assigns injection timestamps to every message in the workload. The schedule respects three invariants: (a) no node's elastic input buffer (Section 4.3) is overcommitted, (b) every spine link's aggregate bandwidth stays within the sizing rule of Section 2.1, and (c) every message completes within its closed-form latency budget. For MoE workloads, the schedule is parametric in the token-to-expert mapping—the router chip fills in the concrete coordinates at runtime via the expert map, but the schedule guarantees that any legal mapping satisfies the buffer credits. For stencil workloads, the schedule is fully static: halo exchanges, compute phases, and result writes are all pre-timed.

The concrete compilation path from a safetensors archive to executing hardware follows a straight pipeline. The ingest stage produces a tensor catalog and computation graph from the safetensors header and model config. The partition stage assigns weight shards to node coordinates via layer mapping, expert round-robin, dense node assignment, and embedding sharding. The map stage assigns operations to kernel names and compute unit types based on tensor roles. The route stage computes routing bitmaps and the MoE expert map. The schedule stage produces a static dispatch timeline. The emit stage writes the .pnm program file and weight binary images. The load stage streams weights to nodes during POST (Section 2.5). The execute stage has the router dispatch tokens and the doorbells fire.

The .pnm program format is the concrete realization of this pipeline: each directive (kernel, bias, token) corresponds to a specific stage's output. A kernel directive encodes the map stage's kernel assignment and the partition stage's weight shard. A bias directive encodes the MAC stub's KERNEL_CONST constant, which the hardware applies to every arriving payload before the resident kernel executes. A token directive encodes the schedule stage's injection: the destina-

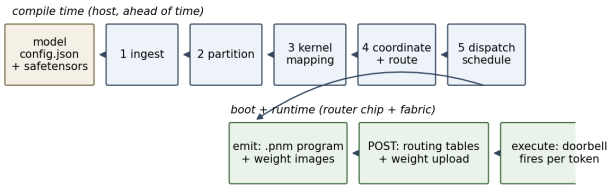


Figure 2: The five-stage ahead-of-time pipeline lowers a safetensors archive and config.json into a static coordinate assignment, routing table, and dispatch schedule; emit, load, and execute complete the path from model file to doorbells firing.

tion coordinate, the payload bytes, and the implicit routing that the route stage computed. The co-simulation harness (Section 2.15) exercises this path end-to-end: the pnmc compiler lowers a plain-text program onto the chassis, generates Verilog stimulus, compiles the fabric slices, simulates them, and verifies hardware-delivered results against the golden model.

A concrete example of the .pnm format. A two-expert program targeting a 512-node chassis: the first expert is a byte-sum kernel on node (board 3, column 2, row 5) with a silicon bias of +7, and the second is a dot-product kernel on node (board 8, column 8, row 8) with bias +3 and a 16-byte resident weight vector. Five tokens are dispatched to both nodes. The .pnm directives encode this as:

kernel sum 2 2 5 bias 7 2 2 5 kernel dot 7 7 7 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f 10 bias 3 7 7 7 token 2 2 5 10 20 30 40 50 60 70 80 90 a0 b0 c0 d0 e0 f0 00 token 2 2 5 01 02 03 04 05 06 07 08 token 7 7 7 de ad be ef token 0 0 0 11 22 33 44 55 66 token 4 3 1 a5 5a 99 00 ff

Each directive line specifies the kernel name, destination coordinates, and—for kernel directives—the resident weight bytes as hex. The compiler lowers these directives into the wire format of Section 2.9: each token becomes a flit carrying [LAYER_ID — MODULE_ID — CTRL — LEN_LO — LEN_HI — payload — CRC_HI — CRC_LO], and the co-simulation harness verifies byte-exact delivery and kernel results against the golden model.

A working implementation of this pipeline exists in the companion repository: the ‘pnmc‘ compiler accepts a HuggingFace model directory (containing ‘config.json‘ and ‘model.safetensors.index.json‘), parses the safetensors header to extract tensor metadata, computes shapes from the model configuration, and performs the five-stage compilation. For example, compiling Google’s Gemma-4-26B-A4B-it (a 30-layer MoE model with 128 experts, 8 active per token, 26.5B parameters in BF16) onto a 4-layer, 4×4 = 64-node chassis assigns 8 model layers per physical layer, distributes 128 experts round-robin across 16 nodes per layer, and produces a chassis schema with coordinate maps, routing bitmaps, and a .pnm program compatible with the co-simulation harness.

The compiler is model-agnostic: it reads any safetensors index and config combination and maps it onto any chassis dimensions. Because every weight is statically assigned to a fixed coordinate and every route is a deterministic function of that coordinate, the compiler has already verified—before execution—that weight dimensions fit the node, that each route is valid, that latency budgets are sufficient, and that no buffer will overflow; the hardware merely executes what the compiler proved correct.

2.8 Coordinate routing and dispatch

The compilation pipeline operates in two phases.

Ahead-of-Time (AOT) mapping. The compiler assigns weights, kernels, and grid subdomains to physical coordinates, then computes every route and latency in closed form: within a layer, X-then-Y between coordinates; across layers, a monotone spine traversal between layer attachments, bracketed by on-board paths. Both are $O(1)$ arithmetic functions of the [Layer ID — Module ID] pair—on a single-path tree there is exactly one route, so no pathfinding algorithm has anything to find. The host-side driver orchestrates this mapping: it precomputes the 11-bit routing bitmap for every node (Section 2.7, Stage 4), builds a MoE expert map that associates each (model_layer, expert_idx) pair with the physical NodeID holding that expert’s weights, and generates the weight upload commands sorted lexicographically by (L, X, Y) for deterministic ordering. Each upload command carries the target Layer ID, Module ID (the byte $X[3:0] \parallel Y[3:0]$), payload length, weight bytes, the compute unit type, and the data type, enabling the host to verify weight upload integrity per unit type. The driver also computes the 11-bit routing bitmap for the router chip itself, which is assigned a sentinel coordinate $(-1, -1, -1)$ and is the only node not loaded into the fabric’s routing tables.

Just-in-Time (JIT) dispatch. Dynamic tokens—such as MoE expert routing decisions—arrive at the central router bridge, which prepends a strict [Layer ID — Module ID] header and injects. There is no runtime arbitration, no congestion sensing, and no adaptive routing. The router’s 1 GHz dispatch stage issues one routing decision per clock, but the sustained dispatch rate is set by the fabric, not the decision stage: injection is bounded by the destination buffer credit rule of Section 4.3, cross-layer traffic by the spine sizing rule of Section 2.1, and per-link traffic by the 16 GB/s link rate. At the reference spine of Section 2.1 the sustained cross-layer dispatch rate is therefore on the order of 10^8–10^9 dispatches/s depending on expert placement—corresponding, at 270 dispatches per generated token (Section 2.6), to a token rate of roughly 10^6 tokens/s—comparable to what 262 TB/s of node-local memory can serve, and far below the router’s raw decision capacity. Memory latency does not move this estimate: the companion benchmark’s memory analyzer evaluates each memory class as a bandwidth-bound rate (chassis aggregate bandwidth times “(1–refresh)” divided by the ≈10

KB of traffic a token generates) and as a strict serial floor that charges each of the 30 decode layers one read-to-first-data wait. LPDDR6, LPDDR5X, LPDDR5, and the DDR5 baseline all clear the honest $\alpha = 7/8$ anchor of $\approx 1.1 \times 10^6$ tok/s by 1.5–2$\times$ (read-to-first-data $\approx$15–20 ns, projections under the validation scope of Section 4.4), so the fabric—not the memory class—sets the ceiling; only high-latency off-node classes (HBM3E's $\approx$100 ns, GDDR6's $\approx$50 ns) fall below the anchor and would bind. Bandwidth never binds at any class: the aggregate node-local bus (262 TB/s at the reference) swamps the kilobyte-class per-token traffic by orders of magnitude. Optional per-layer dispatch replicas scale the decision rate linearly and remain deterministic because the routing function is pure; they do not raise the fabric-bound ceiling.

2.9 The doorbell mechanism

A flit's lifecycle exemplifies zero-overhead activation. The central router chip injects the payload into the fabric. The combinational comparator inside the lxy_repeater at the target layer gates the flit onto the X/Y NoB in sub-nanosecond time. Hardware Flit Repeaters pass the byte stream via DMA into the target node's local address space. The message header carries a length field and a CRC-protected destination coordinate; the node's DMA engine—the memory controller's DMA engine, Section 2.16—counts incoming bytes, computes the running message CRC over the protected body, and compares the protected destination to its own coordinate. When the byte count equals the length field, the CRC validates, and the destination matches, the DMA engine asserts DOORBELL_TRIG and DOORBELL_ACK. This physical pin transition wakes the DUV MAC ASIC instantly; there is no software interrupt handler, no software polling loop, and no operating-system context switch.

The hardware fabric decouples transport from computation. The deterministic repeater network connects to a modular Processing Element (PE) interface via standard AXI-Stream / Ready-Valid handshaking, allowing the underlying execution block (systolic array, FP16 MAC array, or ALU) to be swapped independently of the routing fabric. The companion PE tile stub in the HDL sketch (HDL/core/pe_tile_stub.v) implements this contract: it consumes a flit on its slave port, holds it for one to two clock cycles to model multiply-accumulate latency, and re-presents it on the master port, so the transport-level verification of Sections 3.4 and 4.3 is independent of which execution block sits behind the interface. The stub also computes its resident kernel in silicon—an element-wise bias add—and recomputes the end-to-end CRC over the transformed body, so the co-simulation (Section 2.15) cross-checks the hardware verdict against the software twin on every message. The production compute datapath behind the same interface is the selectable MAC node of Section 2.20, which executes a real MoE-expert pass; the stub is retained precisely because its trivial, arithmetic-independent kernel keeps the transport proof clean.

Every node—whether a MAC node or a repeater serving a CAMM slot—runs the same hardwired three-phase doorbell discipline (Figure 3). This is not software polling: no instructions are fetched and no program counter exists. The phases are the armed and drain states of a finite state machine burned into the doorbell logic:

WATCH (initial phase): continuously check the sentinel bit at the end address of the DMA target buffer. When the sentinel is set (byte count equals header length, CRC valid, and protected destination matches), interrupt WATCH and enter DISPATCH.

DISPATCH: fire the node's resident function. COMPUTE (PNM processor): execute the resident kernel, e.g., matrix multiplication over the landed payload and local weights. FORWARD (repeater at the CAMM slot): stream the payload onward to the next statically mapped node. DOORBELL_ACK is asserted with DOORBELL_TRIG at the fire edge.

DRAIN (reverse phase): while results egress, check the sentinel; when transfer completes, clear the sentinel (bit now empty) and clear the initial payload data; restart WATCH.

Result egress is the same ASIC, not a separate chip: after the kernel completes, the transmit DMA—the reverse half of the doorbell's DMA engine—reads the result from the MAC staging buffer, assembles a return flit whose DEST is the requester (a coordinate the AOT schedule fixes at compile time, Section 2.8), appends the end-to-end CRC, and streams it out the AXI-Stream master port into the fabric (Table 4); the fabric's stateless gates then forward it exactly like any other flit, so the return path adds no new silicon.

The buffer is rigorously empty before the next message can arrive, making back-to-back activations safe without software synchronization. Because every route is a single path of strictly in-order HFR stages, byte-count equality is a sound completion test. An aborted stream always fails the count, the CRC, or the destination check, so the doorbell never fires on a partial, corrupt, or misdelivered payload—and when it refuses, DOORBELL_ACK is withheld, which the source's watchdog converts into an observable NODE_ERR (Section 4.1).

2.10 Reliability and determinism

LPDDR6 provides on-die ECC; the MAC ASIC's memory controller adds conventional SECDED on the bus. Every message carries an end-to-end CRC-16/CCITT-FALSE check over the protected body—[CTRL, DEST, LEN, payload]—and because the DEST field (MODULE_ID) is inside that coverage, a bit error that would corrupt a route decision is caught when the destination DMA refuses the doorbell (Section 2.9); there is no per-link CRC and no mid-fabric discard—a corrupt flit is dropped only at its destination doorbell, and the withheld ACK converts the loss into an observable NODE_ERR

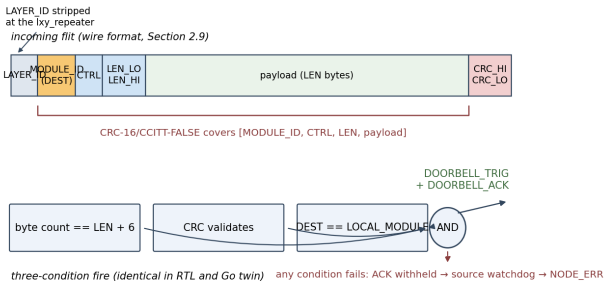


Figure 3: The wire format and the three-condition doorbell. The CRC covers [MODULE_ID, CTRL, LEN, payload] — everything except LAYER_ID and the CRC bytes — so the destination field is inside CRC coverage; only a complete, valid, correctly addressed message fires DOORBELL_TRIG and DOORBELL_ACK.

(Section 4.1).

Deterministic replay is claimed precisely, in two distinct senses. First, re-injecting an identical stimulus onto an identical fabric reproduces delivery bit-exactly: the fabric has no hidden state, and this property is machine-checked in the co-simulation (Sections 3.5 and 4.3). Second, DRAM contents are not assumed to be bit-stable across arbitrarily long runs: LPDDR6 on-die ECC plus the bus SECDED protect every access, and a background scrub pass re-reads resident weights at a rate sized so the mean time to an undetected miscorrection exceeds the chassis lifetime. Post-fault recovery (Section 4.1) re-runs the work on a healthy replica; that is functional recomputation under the same doorbell discipline, not bit-exact replay of the failed node's memory contents. The machine guarantees determinism of the fabric and of computation given a memory snapshot; soft errors in memory are handled statistically by scrub and retry, not claimed away.

2.11 Target workloads

2.11.1 Workload simulation suite

Five canonical workloads exercise distinct routing patterns against the co-simulation harness of Section 2.15, serving as both correctness tests and routing-pattern demonstrations. Each is generated as a .pnm program (Section 2.7) via the pnmc workload command and verified byte-exact on the gate-level fabric.

jacobi5 maps a 5-point Jacobi stencil onto a single physical layer's X/Y grid: each node holds one grid point with stencil coefficients [w_c, w_n, w_s, w_e, w_w] as resident dot-kernel weights in Q6 fixed-point, and each token packs the 5-byte neighborhood [center, north, south, east, west]. Routing is pure intra-layer dimension-order X→Y—zero spine traffic, exactly matching the halo-exchange pattern of Section 2.13.

matvec distributes a matrix row-per-node across all layers: each node's resident dot kernel carries one row's weights (deterministic entries $(i+j) \bmod 251 + 1$), and the input vector is dispatched to every row-node simultaneously. Routing uses spine descent for cross-layer rows; within each layer, X→Y dimension-order. This is the weight-stationary pattern that Section 3.2's roofline analysis targets.

reduction exercises reverse-path merge arbitration: leaf nodes on layer 0 send data fragments toward a root node at the top layer, traversing board egress → Y-up → X-up → lxy_repeater → spine ascent. All nodes run the stateless echo kernel so the golden model is order-independent regardless of merge-point round-robin decisions (a production reduction tree would use accum at the root); what the workload proves is that the arbitrated egress path delivers every packet intact under contention.

broadcast distributes identical payloads from the root to all nodes—the POST Phase-3 weight-upload pattern (Section 2.5) at reduced scale. Routing follows spine descent to every layer, then per-layer X→Y fan-out; it stresses spine-descent bandwidth and the worst-case hotspot on the source node's uplink.

nbody generates all-pairs O(N^2) traffic: every node sends a token to every other node. This saturates every link class simultaneously and serves as an upper-bound benchmark only, capped at 64 nodes for simulation tractability.

All five pass the full verification suite (byte-exact delivery, doorbell accounting, kernel correctness, latency bounds) on the default chassis.

2.12 Mixture-of-Experts transformers

Trillion-parameter MoE transformers present a capacity crisis for HBM-based systems: only a sparse subset of experts is active per token, yet the entire parameter set must be accessible[3, 4]. Our architecture distributes expert weights across hundreds of cheap LPDDR6 pools. A one-trillion-parameter model at INT8 occupies 1 TB—eight reference nodes—and at FP16, sixteen. A single 64 TB chassis holds tens of trillions of parameters with room for replication.

The KV cache store/load/evict cycle is machine-checked end-to-end: KV commands ride the fabric as CRC-protected wormhole flits (CTRL opcodes 0xA0 for store, 0xB0 for load), the offload controller evicts the oldest entry when a bank reaches capacity, and a load returns a CRC-complete response flit the node doorbell accepts (Section 2.15). When a token requires a given expert, the central router chip evaluates the coordinate routing function, dispatches the token via wormhole routing to the holding node, and the doorbell fires. The local MAC executes and returns the updated hidden state. Because only the token travels to the expert—the expert's weights stay resident—the cross-layer traffic fraction α of Section 2.1 is a direct consequence of expert placement: with random placement across eight layers, $\alpha = 7/8$ of token dispatches cross the spine, and the honest spine-sizing rule sizes the fabric for exactly that. Three refinements matter

in production: (i) hot-expert replication from offline activation histograms across k nodes, which both cuts latency and—when replicas are placed layer-locally—shrinks α toward the resident dense-component fraction, trading replication cost against spine provisioning; (ii) expert capacity via elastic input buffers sized to at least one full message burst, with compiler-set capacity factors; (iii) dense components (attention projections, embeddings, output head) replicated on dedicated dense nodes per layer under the same doorbell discipline. Idle experts draw no dynamic compute power—only DRAM refresh current.

The compute unit assignment for MoE inference follows a fixed mapping: expert weight-streaming uses BF16 FMA (weight-stationary multiply), attention QKV projections use BF16 systolic MAC arrays, layer normalization uses FP32 ALU (for the division in variance computation), and the gating network's softmax uses FP32 ALU MIN/MAX/CMP operations. The model compiler (Section 2.7) performs this assignment during the kernel mapping stage, and the firmware (Section 2.6) records the unit type in each dispatch record so the host-side driver can verify weight upload integrity per unit type. The reference chassis sustains 30 dense dispatches plus 240 MoE dispatches per generated token at the dispatch rates of Section 2.8, with the BF16 FMA units consuming approximately 2 bytes per FLOP at the 512 GB/s memory bandwidth.

Expert selection itself is not end-to-end trained in this work. The co-simulated dispatch path routes every token through a deterministic token-seeded routing hash—a 64-bit FNV-1a seed over the token bytes and model layer, mixed per expert, reproducible run-to-run for a fixed input—while the RTL's moe_gating unit (Section 2.17) provides the learned-projection datapath (hidden $\times$ router_weights, top-k selection) verified structurally by testbench with hand-loaded weights. The two paths share the selection shape but not the learned behavior: the hash is a demonstrator for dispatch mechanics, and the learned-router sizing and its per-token evaluation latency are projections (Sections 2.5 and 4.4).

The KV cache is distributed across the attention nodes: on every node that holds attention Q tensors, the compiler reserves 80% of that node's post-weight budget for KV entries, capped at 16,384 entries (a 16K context window). Each KV entry occupies $\mathrm{HiddenSize} \times 4$ bytes (K and V, each 2 bytes in BF16, per hidden dimension), bounded below by the co-simulated and RTL cache's fixed 512-byte frame per entry (Section 2.7, sub-phase 2d). Entries are distributed round-robin across four directional banks (X+, X-, Y+, Y-) by sequence position, and eviction is triggered when any bank exceeds the 80% capacity threshold. The firmware checks the KV cache after each attention dispatch: if a bank needs offloading, it dispatches a KV offload command through the spine to host DRAM and triggers a reload cycle before the next token. The KV cache is entirely hardware-managed—no software polling, no OS page faults—and its FIFO eviction policy is deterministic given the same sequence of token arrivals.

For interactive inference (chat, code generation, retrieval-augmented generation), the architecture includes an LLM inference client that handles tokenization, prefill, autoregressive generation, and temperature/nucleus sampling. The client boots the firmware, builds a vocabulary from the model's tokenizer configuration, and generates tokens one at a time: each token triggers a prefill phase (all layers, all experts) followed by an autoregressive phase (cached KV state, reduced compute). The sampling pipeline computes softmax over the logit vector with numerical stability (max-subtraction trick), applies temperature scaling, sorts by probability, and selects via nucleus (top-p) sampling, all running on the host with the results dispatched to the chassis as token directives. The client tracks per-inference statistics—tokens generated, prefill tokens, total dispatches, MoE and dense dispatches, KV store/load/eviction operations—and reports compute unit utilization across the chassis. Token encoding on the wire varies by data type: 2 bytes for BF16 and FP16, 4 bytes for FP32, and 8 bytes for FP64.

2.13 Scientific HPC stencil workflows

Climate modeling, computational fluid dynamics, and seismic imaging rely on regular FP64 stencil operations over massive grids. These workloads map onto the rigid X/Y grid of sockets: each grid subdomain becomes a physical node, and halo exchanges become single-hop dimension-order routes between neighboring sockets that never enter the spine. Because the stencil neighborhood is known at compile time, the AOT compiler generates exact routes with no runtime indirection, replacing cluster middleware such as MPI[19] with a coordinate function.

The roofline analysis of Section 3.2 shows why mature nodes suffice: a 7-point FP64 stencil has an arithmetic intensity of roughly 0.3–0.5 FLOP/byte, far below the reference node's machine balance of about 1.5 FLOP/byte[5]. Where numerics permit, mixed-precision halo storage raises effective intensity further. Deterministic worst-case halo-exchange latency is a compile-time constant (Section 4.4).

2.14 Functional data-based workflows

The mappable subset is first-order dataflow graphs of pure kernels over statically allocated buffers[6, 11]. Within this subset, an actor is a node, a message is a flit, and a statically scheduled functional kernel is a doorbell activation. General lazy graph reduction with an unbounded dynamic heap is future work (the Conclusion section). Referential transparency is exactly what makes the replay guarantees of Sections 2.6 and 4.1 hold.

2.15 Verification methodology: machine-checking the transport claims

The transport claims of this paper are machine-checked, not asserted. The companion repository ships a co-simulation harness—Go, standard library only—that drives a cycle-exact Verilog-2005 model of the fabric: the routing gates of Sections 2.1 and 2.2, the per-node PE tile stub of Section 2.9, and the end-to-end CRC machinery of Section 2.10[28]. The harness is deterministic by construction: it embeds a byte-exact MT19937 clone of the CPython random.Random generator (a Python-equivalent Mersenne Twister implementation verified against CPython's known outputs), so a given seed reproduces the identical stimulus file, generated Verilog, and delivery log on any machine, and every result quoted in Section 3.5 is bit-reproducible.

A software oracle checks five properties on every run: (1) byte-exact delivery and byte conservation—the bytes delivered to each node's DMA port equal the bytes injected for it, and the totals match, so no byte is lost, duplicated, or reordered anywhere in the fabric; (2) zero drops and zero misroutes—every injected flit reaches exactly its declared destination, and the routing-bitmap comparators of Section 2.1 never raise route_err on valid traffic, even under backpressure; (3) doorbell accounting—the hardware verdicts (activations, refusals, and the stub's corrupt_out pulses) match, message by message, the software twin's three-condition fire logic of Section 2.9; (4) kernel correctness—each resident kernel (echo, sum, accum, dot) executes on what the hardware actually delivered and must equal the golden model computed over the payload, including the bias the MAC stub already added in silicon; (5) latency bounds—the sweep scenario checks every packet against the closed form of Section 3.4 exactly (hop count times per-hop delay plus serialization at the link rate, plus the stub's elastic pipe), while the other scenarios assert the pipe floor only, because contention is a compile-time-scheduled term.

Six scenarios exercise the fabric. sweep sends one flit to every node and checks the exact closed-form latency; vcsweep repeats sweep while also injecting pass-through traffic on all four virtual-channel classes of Section 4.3, so a single run exercises the entire VC stack including the arbitrated egress tree; load sends 500 random flits (450 routed to node kernels plus 50 pass-through) to random kernels with 100/50/25% sink backpressure and proves lossless delivery with slow consumers; hotspot concentrates MoE hot-expert traffic (Section 2.12) on a worst-case corner node behind a 1-in-8 DMA gate; stress injects roughly 24 flits per node with 4 B–1 KB payloads, corrupts the end-to-end CRC of approximately 3% of messages—which the doorbell must refuse—and passes approximately 5% through the fabric; replay runs stress twice and requires the two delivery logs to be bit-identical, machine-checking the no-hidden-state claim of Section 2.10. Two opt-in KV scenarios exercise the cache banks of Section 2.12: kv stores CRC-validated entries into the per-column banks and loads them back as CRC-complete compute flits the node doorbell accepts (corrupt commands are drained and refused), and kvflow drives a small bank through fill, automatic eviction by the offload controller at capacity, and reload, machine-checking the circular FIFO window against a software twin that replays the same command sequence with the same eviction rule.

Three hand-written HDL testbenches complement the harness: a functional fabric smoke test, a 500-packet load test with backpressure and per-packet checksums, and a doorbell test that drives eight packets through the PE tile stub and checks the hardened fire conditions—six activations, two refusals (a truncated message and a wrong destination), and two stub corrupt_out pulses on messages whose CRC fails in flight.

Additional testbenches verify every compute unit type in isolation: FP16, BF16, FP32, and FP64 FMA modules (3-cycle pipeline, bit-exact golden-model checks), the FP32 ALU (restoring binary long division, MIN/MAX/CMP, and FMA-path operations), the INT8 MAC (two-cycle multiply-accumulate), the BF16 and FP16 systolic MAC arrays (weight loading, burst activation, zero-weight identity), the MoE gating unit (softmax top-k selection), the host bridge (a byte-exact wormhole flit with its CRC-16, and the dispatch counter), and the host path end to end (SPI adapter, arbiter, and bridge composed, checking a register read-back, the emitted flit, and the result path).

Beyond the fabric and compute silicon, tb_sodimm_ctrl.v verifies the banked memory controller described in Section 2.16—exact row-hit/miss/conflict latencies, tRAS enforcement, byte-enable masking, refresh-storm integrity, and a DDR4-versus-DDR5 ordering sweep—and tb_optical_link.v exercises the optical inter-chassis links of Section 2.19: idle-link latency, burst framing alignment, flood-level losslessness, gapped packets, an asynchronous 100–80 MHz clock crossing, and simultaneous bidirectional traffic.

A Verilator static lint pass confirms the routing fabric is free of multi-top errors and width mismatches across all gate-level modules. The AOT compiler path is exercised end-to-end by a small program compiler that lowers a plain-text program (kernel, bias, and token directives) onto the chassis and runs the identical pipeline; the shipped example places a byte-sum and a dot-product expert with silicon biases on the 512-node chassis and verifies their results on the gates (Section 3.5).

The forward path is a demux tree and makes no arbitration decisions, but the reverse path—node TX to Y-up, X-up, lxy_repeater, and up-spine—is a pure merge tree whose arbitration is load-bearing (Section 4.3). Every merge is a 2-in/1-out module that grants one upstream port for a whole packet—the decision is held until EOP, so packets are never torn—and round-robins between the two ports; the granted port's ready tracks the downstream handshake while the other stalls in place. Because the merge dependency graph is a DAG (board egress toward spine ascent, never a cycle), arbitration cannot deadlock. The harness therefore validates the

egress path twice: in the cycle-exact RTL itself, and through an independent activity-driven discrete-event model (DES) of the merge tree that shares no code with the Verilog. A cross-check oracle compares the DES delivery stream byte-for-byte against a real RTL slice on a mixed four-class program (Section 3.5), so agreement between the two implementations pins the arbitration logic down rather than echoing one implementation against itself.

vvp is single-threaded, so the harness partitions the layer list into contiguous groups—one per CPU core by default—and runs one vvp process per slice in parallel, compiling each slice's Verilog concurrently. The split is byte-exact against a monolithic simulation because the spine stages upstream of a slice's destination layers are transparent pass-through; cross-group spine contention is a compile-time scheduling concern, not a gate-level effect, and is deliberately left uncoupled (the AOT schedule of Section 2.8 assigns it statically). Each slice's testbench injects from a stimulus file, applies per-node backpressure (each DMA sink accepts one byte in N cycles, N fixed per scenario), and terminates adaptively: it finishes 200 cycles after the last transition on any fabric wire, with a watchdog at 64 times the injected byte count for pathological stalls.

2.16 The LPDDR6 memory controller

The memory controller is the ASIC's only memory-facing state machine, and it is the block that turns the physical buses of Section 2.4 into the ≈512 GB/s figure quoted throughout this paper. It sits between the LPDDR6 PHY and the two consumers on the node—the MAC array and the doorbell DMA engine—and it is hardwired, not programmable: its policy is a fixed function of its inputs, which is what lets the compiler's static schedule (Section 2.8) treat memory service as a deterministic resource. Its place in the ASIC, pin group, and power budget are tabulated in Section 3.6 (Table 4).

Bus structure. Each 192-bit interface is four 48-bit channel groups, i.e. eight 24-bit LPDDR6 channels of two 12-bit sub-channels each—sixteen independent sub-channels on one module. The reference node pairs two modules, so its controller commands thirty-two independent sub-channels across a 384-bit aggregate. Each sub-channel owns its own bank array, command path, and strobes, so the controller can keep up to thirty-two rows open simultaneously and interleave transfers across them. This independence is the entire trick: with thirty-two sub-channels in flight, the controller hides the two inherently serial DRAM operations—row activation and precharge—behind transfers on the others, sustaining full bus rate while no individual bank is ever busy more than a fraction of the time.

Access flow. The dominant operation is a long read burst: the MAC array consumes resident weights at the bus rate, and the AOT compiler lays weights out contiguously (Sections 2.7 and 2.8), so each row activation serves a long run of column reads and the row-buffer hit rate approaches 100% in steady state. The controller's bank scheduler issues activate, read/write, and precharge commands to the sub-channels in rotation, batching the residual random accesses—KV-cache updates and result writes—into write slots between weight streams. A bounded reordering window is safe because the fabric's closed-form latency bounds terminate at the node's DMA port (Sections 3.4 and 4.3); kernel completion timing is not part of the verified bound, and results are byte-exact regardless of scheduling. The elastic input buffer of Table 4 (one or more maximum-length messages) decouples the MAC's demand from DRAM timing on both sides of the pipeline.

Refresh and ECC. The controller reserves statically sized refresh slots in the schedule—deterministic by construction, so the compiler can size the elastic buffers to cover them—and the isothermal manifold of Section 2.3 keeps the DRAM at $\leq$70°C, below the high-temperature refresh regime. Error protection is layered exactly as Section 2.10 specifies: LPDDR6 on-die ECC corrects single-bit errors inside the DRAM, and the controller's SECDED code over the bus catches everything between the die and the ASIC, computed inline in the PHY datapath with no added pipeline latency. The background scrub pass of Section 2.10 issues periodic read-and-verify cycles through the same bank scheduler at low priority; a scrub read never preempts a kernel's weight stream, and its rate is sized so the mean time to an undetected mis-correction exceeds the chassis lifetime.

Doorbell DMA path. The doorbell's DMA engine of Section 2.9 lives inside the controller. Incoming flits are written into the target buffer as they arrive through the write path; the DMA engine maintains the byte count, computes the running CRC over the protected body, and compares the protected destination—all inline in the controller datapath. When the count reaches the length field, the CRC validates, and the destination matches, the controller sets the sentinel bit at the end address in the same cycle the doorbell fires (Section 2.9). The WATCH-phase sentinel check is a single read issued through the normal scheduler; it costs one bank access every few hundred cycles, negligible against the weight stream. The same write path loads weights at boot: the POST sequence of Section 2.5 streams 64 TB through the host PCIe uplinks in approximately 12–15 minutes as a long, purely sequential write burst per node—the controller's easiest workload—with upload time bounded by the host uplink, not the fabric (Section 4.4).

Determinism. The controller holds timing state (refresh counters, bank timers, scheduler pointers) but no data state, and its policy is deterministic given its inputs, so it introduces no non-determinism into delivery or results: identical stimuli onto an identical chassis reproduce identical computation given a memory snapshot, exactly as Section 2.10 claims. Residual DRAM timing variation is absorbed by the elastic buffers and never perturbs the data path, and soft errors are handled statistically by ECC, scrub, and the recompute protocol of Section 4.1—not claimed away.

Generation portability. The controller's bus contract—valid/ready acceptance, CAS-latency wait, one response per transaction—is deliberately the only interface its consumers see, which makes it portable across DRAM generations. Three behavioral PHY variants in the repository pin this down: an LPDDR5 profile (four banks, CAS 16, tRCD 4, tRP 3, tRAS 8, 3.9 μs refresh), an LPDDR5X profile (CAS 14, tRCD 3, tRP 2, tRAS 7), and the flagship LPDDR6 flat-bank profile (CAS 4). Each is exercised by a self-checking testbench that polls the ready strobe rather than assuming fixed delays (tb_lpddr5_phy.v, tb_lpddr5x_phy.v, tb_lpddr6_phy.v). A deployment selects the module class by synthesis-time parameter; the MAC array, doorbell DMA, and firmware are untouched—the same generation-agnostic claim Section 3.7 machine-checks for SODIMM classes, extended to the LPDDR5/5X/6 CAMM2 ladder.

2.17 The router bridge: a CPU-less chip with a host link

The router bridge described in Section 2.5 has no processor of its own. The chip is a bridge between the host and the fabric: it performs Power-On Self-Test and topology discovery, accepts the host's routing tables, weight blobs, and dispatch commands over a host link, evaluates the coordinate mapping for dynamic MoE expert routing, and injects every JIT-dispatched token into the fabric. All sequencing—the five boot phases of Section 2.5, the dispatch loop, the KV-cache policy—lives in the host's firmware, which the repository ships in two forms: a Go reference model (the co-simulation's golden implementation) and a C port with static allocation only, no `malloc`, and no operating-system dependency, so it runs on a Raspberry Pi, on a microcontroller, or on a workstation without change.

The bridge is the manager for the chassis as a whole, but not necessarily for each gating ASIC: the routing engines on the node boards can be managed locally instead (Section 2.17.1).

This placement is a deliberate consequence of where the work actually is. The bridge's data path is fixed-function and needs no programmability: the flit builder and the CRC-16/CCITT-FALSE are hardware, and the host writes their operands through a small register window. What the bridge does need is sequencing, and sequencing is exactly what a host already has. Keeping the processor off the bridge therefore removes a whole class of silicon—the boot ROM, the timer, the console, the interrupt controller, and the memory-management assumptions an OS would impose—without removing any function the fabric needs, and it makes the bridge attachable to whatever the deployment already has: a Raspberry Pi over SPI, or any third-party microcontroller over a byte-level host link with the same register and framing semantics.

The host link is deliberately the simplest thing that can carry the job. The bridge exposes a 32-bit register window and a byte-stream flit path; a host writes the routing tables and the weight blobs, triggers an injection, and reads back the status counters and the extracted result bytes. The repository's SPI adapter (pi_bridge.v) is one instantiation of that link for a Raspberry Pi, and the C port of the firmware is the host-side counterpart. The protocol is a sequence of register writes and a framed byte stream—no descriptor rings, no interrupts, no cache coherency, and nothing that would need an OS to drive it.

For source-language transpilation (Section 2.7), the main thread runs on the host, not on any node PE. The host compiles R/Haskell/HLSL source to typed IR (the existing CompileR/CompileHaskell/CompileHLSL pipeline), and the host's firmware schedules the resulting program over the host link. This placement follows from three constraints. First, the compiler's IR lowering stage needs visibility into the full chassis topology—which nodes hold which compute units, which routing patterns connect them—and only the host-side compiler has it. Second, the lowering is a compiler, and compilers run where the source language's toolchain runs. Third, keeping the transpiler off the bridge means the bridge's silicon stays a fixed function: adding a hosted-language runtime would make the bridge a general-purpose processor again, which is precisely what this design removes.

The register window is small and flat: the flit-builder command registers (destination, length, payload byte, trigger), the status and counter registers, the result byte, and a 64 KB staging SRAM. The host copies weight bytes through the window; the behavioral LPCAMM2/LPDDR6 module model (lpddr6_camm.v) is exercised by the node-local testbenches, and a DMA engine onto it is not yet part of the bridge.

The design is verified by a self-checking testbench (tb_pnm_host_bridge.v): the testbench plays the host, writes the routing and payload registers, and checks that the chip emits a byte-exact wormhole flit—header, payload, CRC-16 over the protected body, and SOP/EOP framing—and that the result path and the dispatch counters read back correctly. The host link itself is verified by a second testbench (tb_host_bmc.v) that drives the same bridge from a Raspberry Pi model over the SPI adapter, through the two-port arbiter (pnm_arb), and checks that the register writes land and the responses return.

2.17.1 Two managers for the gating ASIC

The gating ASIC is a per-node device: it holds the router weights and produces the expert decision for each token that lands on its node. Who manages it is a board-level choice, and the silicon supports both because the decision path and the management path are separate.

In mode A an embedded management node sits on the node board: an MCU or a Raspberry Pi Compute Module on a mezzanine header beside the gating ASIC. It drives the gating ASIC's register port directly, and it is also a master of the

node's NoB, so it can reach the memory behind the node. It therefore programs the gating weights once during machine setup, then reads each token's hidden state out of local DRAM, presents it to the gating ASIC, and takes the decision locally. The per-token routing decision never leaves the board: it costs a local register write, a local memory read, and a local register read, and the token's data path is untouched. The node's firmware, the board's glue between the management node, the gating ASIC, and the fabric, and the decision's read-back are all machine-checked (Section 3.5).

In mode B the host owns the whole machine. The host's gating programming is streamed over the fabric to the gating ASIC, whose fabric-side receiver decodes it into the same register map the embedded node uses. This is the mode the host-driven bridge of Section 2.17 is built for, and it is the flexible one: the host can reprogram the router of any node at any time, and the same host firmware drives a Pi, a microcontroller, or a workstation. The shipped receiver is write-only: the programming path is complete, and the fabric-side read-back of the decision is the remaining piece of work, so a mode-B manager today reads the decision from the chip's local output wires rather than over the fabric.

The two modes are the same die. A strapping pin on the node board selects the manager, and the register map, the weight format, the top-k selection, and the decision values are identical either way; a self-checking testbench (tb_gating_modes.v) programs the same weights and the same hidden state through each path and requires the same decision.

The difference is in what crosses the fabric. In mode A the gating weights never touch it: the management node loads them into the ASIC once, locally, and each token's decision is a local register read. In mode B the host must push the 0.7 MB of weights across once per layer at setup and, thereafter, push each token's 5.6 KB hidden vector in per layer, with the decision returned to the host. Against the roughly 2.2 MB a token's own dispatches generate (Section 2.1), that per-token management traffic is under ten percent: the honest case for mode A is architectural rather than bandwidth-critical—the decision is made and acted on where the token already is, so it never becomes fabric traffic at all, and the node does not depend on the host to route it.

2.18 PCB interconnect models

The board-level wiring that Sections 2.3 and 2.4 describe qualitatively—spine mezzanine jumps, motherboard point-to-point traces, CAMM2 socket escapes—is modeled in the repository as parametric timed wires (pcb_link.v). Each link is a shift-register delay line of $\lceil \text{DELAY_NS}/10 \rceil$ reference-clock cycles: data, valid, and the start/end-of-packet markers traverse bit-exactly, arriving a deterministic number of cycles later, and nothing else happens—no protocol conversion, no buffering, no reordering. Drive-current and trace-impedance parameters ride along as documentation of the electrical envelope each profile assumes. Three profiles cover the chassis's three PCB populations: the spine mezzanine connector (≈5 mm mated, 128-bit), the motherboard point-to-point trace between repeater and node (≈40 mm, 128-bit), and the CAMM2 socket escapes to the MAC ASIC PHY (two ≈20 mm runs, 192-bit each). A wrapper (pcb_triple_link) instantiates three links sharing one clock domain for the X, Y, and spine egress fans a node actually has, with an aggregate flit counter for activity accounting.

tb_pcb_link.v pins the model down with four checks: a single-flit passthrough preserving SOP/EOP and payload; an exact-latency check that a 40 ns profile delivers in precisely its four configured stages, not approximately; a 200-packet randomized stream (1,408 bytes, per-packet lengths of 1–12 words) delivered byte-exact and in order against a scoreboard; and the triple-link aggregate counter matching an independent injection count. These bounds are what let Section 3.4 add per-population wire delay into the closed-form latency expression as a compile-time constant rather than a statistical spread: each populated segment contributes exactly its configured delay, and the co-simulation's hop accounting already treats every such segment as one cycle of the closed form.

2.19 Inter-chassis optical links

The scale-out path of Section 3.7 leaves the chassis through the spine root, and its physical realization is a fiber-optic inter-chassis link pair (optical_link.v). The link is deliberately protocol-transparent: it moves an uninterpreted byte stream with start/end-of-packet delimiters, so the fabric wire format of Section 2.9 crosses the fiber untouched and neither endpoint rewrites a flit. Each unidirectional link is a dual-clock elastic buffer whose write and read pointers cross the domain boundary as Gray codes through two-flop synchronizers, followed by a propagation pipeline that models electrical-to-optical conversion, fiber transit at the speed of light in glass ($n \approx 1.47$, approximately 4.9 ns per meter), and optical-to-electrical recovery, quantized up to whole reference-clock cycles. A wrapper composes two links into one bidirectional chassis-to-chassis port—one fiber per direction, as an optical transceiver is actually wired.

Flow control across the fiber is structural rather than probabilistic. The read domain pops whenever the buffer is non-empty—no return handshake ever crosses the clock boundary—and all backpressure is applied on the write side, where transmit ready deasserts once the Gray-synchronized occupancy reaches buffer depth minus the propagation pipeline minus a guard term. Any producer that respects ready therefore cannot overflow, and a protocol violation latches a sticky error flag instead of silently dropping bytes. Idle-link latency measures propagation depth plus five cycles at the 100 MHz reference (synchronizer visibility plus the output register), which for the default two-meter board-to-board span budgets roughly 70 ns end to end—about one intra-chassis spine hop per meter of fiber, so a multi-chassis rack adds single-digit hop-equivalents to the closed-form latency of

Section 3.4. Because an optical hop adds conversion and serialization error sources mid-fabric, the receiving router re-runs the CRC-16 check of Section 2.9 on arrival (the shared crc16 engine) and refuses corrupt frames exactly as a node doorbell would, keeping silent corruption from propagating into a second chassis. A six-scenario testbench (tb_optical_link.v) pins the model down: exact idle-link latency, 200-word burst framing alignment against a scoreboard, a 600-word back-to-back flood delivered with zero loss, sparse packets separated by idle gaps and held in order, an asynchronous 100 MHz-to-80 MHz clock-domain crossing under sustained overload that observes transmit ready deassert, and simultaneous bidirectional traffic across the pair.

2.20 The compute node and the gating ASIC

The PE interface of Section 2.9 is a contract, and the repository instantiates it twice: a transport stub whose trivial kernel keeps the CRC cross-check independent of the arithmetic, and a real compute datapath used for the numerical claims. The real node (HDL/core/node.v) is a MAC ASIC that runs one Mixture-of-Experts expert pass end to end on the same BF16 fused-multiply-add datapath as the router—gate and up projections over the hidden state, a gated linear unit act = gelu_tanh(gate)·up, and the down projection—accumulating with a single rounding per multiply-accumulate, the same oracle the router logits use. The node is not specialised to one numeric format: a CU_TYPE parameter selects the embedded accumulate engine (BF16/FP16/FP32/FP64 FMA, or INT8/INT4/FP4 MAC with a block-scale input for MXFP4) behind one shared engine interface and one control finite state machine, so a single node source builds as any of the compute units of Table 5 and the numeric paths cannot drift apart in their control logic. The node reads its hidden state from the per-node multi-channel memory subsystem (HDL/core/node_mem.v): the linear word space is word-interleaved across CHANNELS banks, so a request for N words returns $N/\mathrm{CHANNELS}$ in-order beats of CHANNELS words each after a fixed CAS latency. That interleave is the RTL mechanism behind the per-node bandwidth scaling of Section 2.16 and the dual-module 384-bit packaging of Section 2.4 (CHANNELS=2 for the reference node), and the expert result is invariant under the channel count—only the ingest width changes.

The gating network runs on its own ASIC beside the node. moe_gating.v computes the router logits (hidden $\times$ router weights, again with fused-FMA accumulation) and selects the top-K experts; gating_asic.v wraps it in a register bus that exposes the weight- and hidden-SRAM write ports, the start/done handshake, a rank window that returns every top-k id and logit, and a logit-peek window that returns any of the 128 expert logits, so the manager reads the full routing decision back over the register interface rather than re-deriving it in software. That bus has two masters (Section 2.17.1): the embedded management node on the node board drives it locally, or the host's programming arrives over the fabric and is decoded into the same register map, and a strapping pin on the board chooses between them.

These datapaths are validated bit-exact, not merely sketched. A complete decoder block (HDL/core/gemma_decoder_block.v)—input and attention normalisation, Q/K/V projections with per-head QK and value norms, rotary position embedding, causal attention over a KV cache, the output projection with its residual, the dense feed-forward pass, and the top-2 MoE branch with router softmax and per-expert scaling—is instantiated from a real layer-0 slice of the Gemma-4-26B-A4B checkpoint and reproduces the reference BF16 forward pass word for word in gate-level simulation. At the system level the three stages compose into one datapath: the gating ASIC's selected expert is dispatched as a single fabric flit (delivered byte-exact through the node-network co-simulation of Section 2.15), the destination compute node runs that expert on its selectable ASIC, and the node's output matches an independent software BF16 twin on every word. The composition is chained rather than sampled: over model layers 0, 7, and 14, the router input of each layer is derived from the residual slice left by the previous layer's node, so the gating decision, the fabric dispatch, and the expert recomputation form a single data dependency across layers, and each layer's node output is again bit-exact against the software twin (the chain runs at a reduced 256-dimension slice, so the router sees the matching 256-column weight slice, to stay inside a cycle budget). Closing the loop, feeding the RTL router's logits into the engine in place of the software matvec reproduces the software-only inference token for token—so the hardware gating decision changes no output. All of these checks run on real checkpoint weights and are part of the co-simulation suite of Section 3.5, which closes the gap between the transport properties the paper machine-checks and the computation those transports carry.

Results

Cost, power, and thermal figures in this section are first-order engineering estimates intended to establish plausibility and sizing rules, not measured results; no physical prototype exists yet. The transport-level claims are machine-checked, not estimated: Section 3.5 reports the co-simulation verification results against the cycle-exact fabric model of Section 2.15.

3.1 Reference chassis

Table 2 summarizes the reference configuration: 512 nodes, 64 TB aggregate capacity, approximately 262 TB/s aggregate node-local bandwidth, approximately 197 TFLOPS FP64, and 8–10 kW chassis power, with a spine provisioned at $\approx$2 TB/s per direction from the cross-layer sizing rule of Section 2.1. The configuration is defined by three physi-

Table 2: Reference chassis configuration (first-order estimates).

Quantity	Reference value
Node	2 × 64 GB LPDDR6 CAMM2 (128 GB, 384-bit) + 14 nm DUV MAC ASIC (192 FP64 FMA @ 1 GHz ≈384 GFLOPS FP64; ≈768 GFLOPS FP16; ≈1.5 TOPS INT8)
Node memory bandwidth	≈512 GB/s (2 × 192-bit LPDDR6 CAMM2 @ 10.7 Gb/s/pin, 384-bit)
DDR5 baseline (contrast)	192 GB dual-slot (2×96 GB DDR5-5600 SODIMM); ≈90 GB/s per node—≈3× below the LPDDR6 reference
Node fabric link	≈16 GB/s (128-bit source-synchronous @ 1 GHz)
Node power	≈15 W
Board (1 layer)	8×8 = 64 nodes; 8 TB; ≈16 TB/s; ≈1 kW
Chassis (8 layers)	512 nodes; 64 TB; ≈262 TB/s; ≈197 TFLOPS FP64; ≈8–10 kW
Spine	Parallel 16 GB/s lanes; ≈2 TB/s per direction aggregate (cross-layer sizing rule, Section 2.1)

cal properties that no market correction can alter: (a) no interposer dependency—each node is a standard JEDEC CAMM2 module paired with a socketed DUV ASIC, with no advanced packaging between them; (b) low thermal density—approximately 15 W per node versus 300 W for an H100-class GPU, enabling air or liquid cooling without exotic thermal solutions; and (c) mature-node fabrication—the 14 nm DUV MAC ASIC benefits from yields above 90% on established process lines, while the HBM stacks it pairs with consume roughly three times the wafer area per bit of commodity DRAM and absorb a disproportionate share of leading-edge fab capacity[25]. Cost is a consequence of these physical properties: commodity DRAM in volume, mature-node silicon, and standard packaging yield a node at approximately $2,000, but the structural advantage exists independently of DRAM spot pricing.

The DDR5 baseline row contrasts the reference node with the commodity SODIMM deployment of Section 3.7: the same 14 nm ASIC, repeater, doorbell, and firmware against a dual-slot DDR5-5600 SODIMM pair (≈90 GB/s, 192 GB). Section 3.2's roofline places both flagship workloads below even that reduced ridge point—weight streaming at 1–2 FLOP/byte is bandwidth-bound on DDR5 exactly as on LPDDR6—so the bandwidth-bound conclusion is generation-independent, not an artifact of the LPDDR6 choice. The reference chassis simply selects the highest-bandwidth socketable module class for the same silicon.

3.1.1 The reference chassis as a dedicated capacity-serving appliance

At full scale the 512-node, 64 TB chassis is positioned strictly as a serving appliance for capacity-bound, weight-stationary workloads—Mixture-of-Experts transformer inference chief among them[27]—and the placement tooling makes that positioning concrete rather than aspirational. Compiling an MoE transformer onto the 8×8×8 chassis emits three artifacts: a routing table of exactly 512 entries—one 11-bit bitmap per node (the per-layer repeater bitmaps ride the same table), loaded during POST; an expert map placing all experts of every model layer across the expert-parallel node pool with no data-parallel replication waste; and a per-token dispatch plan. For the Gemma-4-class reference model fixture (30 layers, 128 experts, top-8), the plan is 270 dispatches per token: 30 dense attention projections plus 240 expert activations, each carrying its destination coordinate, compute-unit assignment, KV-cache disposition, and flit byte count. The firmware boot sequence is verified against the same artifacts: five phases (POST discovery finding all 512 nodes, routing-table load of 512 bitmaps, weight upload of 8,045 integrity-checked commands, MoE gating load of 3,840 expert mappings—128 experts × 30 layers—and ready) complete with zero error counters, and the dispatch plan verifies coverage: every activated expert maps to a resident node within budget.

The capacity arithmetic is the point of the configuration. Each node reserves up to 80% of its post-weight budget for KV offload, capped at 16K entries per resident layer so that context growth evicts cleanly instead of fragmenting; the remaining capacity holds expert weights at BF16. Expert placement is deterministic but not round-robin at runtime: the compiler assigns each expert to exactly one node (Section 2.7's subphase 2b distributes experts round-robin over the node pool at compile time), and the gating network—evaluated per token by the router—selects which experts fire, so a token's top-8 fan-out touches eight nodes plus one dense node per layer, and the closed-form route of Section 2.8 bounds each hop regardless of which experts the gate selects. Nothing in the configuration is specific to this model family: the compiler re-derives placements, bitmaps, and dispatch plans from any config.json/safetensors pair, which is what makes the appliance a configuration of the architecture rather than a distinct machine.

3.2 Roofline: why mature nodes suffice

The reference node's machine balance is precision-dependent, because the DUV MAC array packs INT8 and FP16 lanes at 4× and 2× the FP64 rate (Figure 4):

$$B_{\text{machine}}^{(\text{FP64})} = \frac{384\ \text{GFLOP/s}}{256\ \text{GB/s}} \approx 1.5\ \text{FLOP/byte},$$
$$B_{\text{machine}}^{(\text{FP16})} \approx 3.0,$$
$$B_{\text{machine}}^{(\text{INT8})} \approx 6.0.$$

A 7-point FP64 stencil has arithmetic intensity ≈0.3–0.5 FLOP/byte, below the 1.5 FP64 balance (strongly bandwidth-bound)[5]. MoE expert weight-streaming sits near 1–2 FLOP/byte per token—below the 3.0 FP16 balance and far below the 6.0 INT8 balance, so MoE is bandwidth-bound on the precision it actually runs, even at the top of its intensity range. Performance is therefore limited by the LPDDR6 bus,

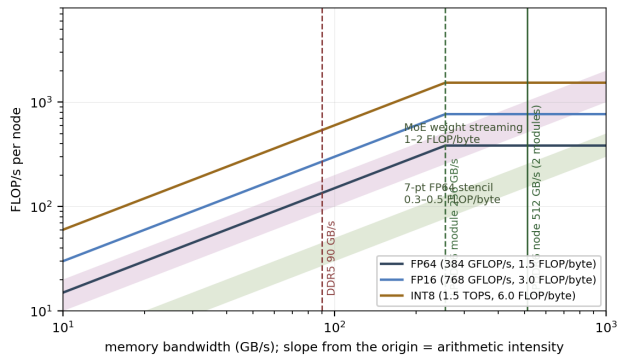


Figure 4: Roofline for the reference node. Both flagship workload intensities fall below all three machine-balance ridges (FP64 1.5, FP16 3.0, INT8 6.0 FLOP/byte), so LPDDR6 bandwidth — and even the DDR5 baseline — binds before the MAC array does.

never by the MAC array, for both flagship workloads. This is the load-bearing argument for DUV: a 14 nm or 28 nm array already saturates the memory it is attached to.

3.3 Cost and capacity model

The cost advantage is structural, not cyclical. A node bill of materials: LPDDR6 CAMM2 module ≈$1,900 (≈$15/GB at 2025–26 DRAM spot pricing), 14 nm MAC ASIC ≈$40, socket/board/assembly share ≈$60, for a node total of ≈$2,000 (≈$16/GB). A 512-node chassis is ≈$1M for 64 TB. The comparison to HBM is capacity-equivalent, not bandwidth-equivalent: a 64 TB HBM system requires 819 H100-class GPUs (80 GB each)—≈$24.6M of accelerators at ≈$30k apiece, ≈0.25 MW of GPU power alone—against ≈$1M, ≈10 kW, and one chassis for the same 64 TB here. Even the HBM itself, at ≈$375/GB[2], costs ≈$24M for 64 TB: 24× the entire PNM chassis, memory included. The HBM fleet's aggregate bandwidth dwarfs the 262 TB/s of node-local LPDDR6, but Section 4.5 shows inference serving is capacity-bound rather than bandwidth-bound, so that excess bandwidth idles during decode while the capacity bill is paid in full. DRAM is a cyclical commodity[25], and the 2025–26 spot prices used here reflect a shortage-driven spike; but the structural factors—commodity modules versus interposer-bound stacks, mature-node DUV versus EUV, standard packaging versus advanced integration—persist across market cycles. The advantage is architectural, not a bet on a particular price point.

The comparison deliberately spans only DRAM- and HBM-resident systems. The SRAM-resident accelerators surveyed in Section 1.5 are capacity-infeasible at 64 TB by construction, and the mask-ROM MSIC reaches similar capacity economics only by freezing the model in silicon—the reprogrammability this design preserves (Section 1.5). The strongest outside validation of the capacity-bound decode thesis is commercial: both major GPU vendors have since acquired or licensed an SRAM or mask-ROM inference shop rather than build against the HBM wall directly.

3.4 Thermal and latency budgets

Thermal: ≈15 W per node, ≈1 kW per board, ≈8–10 kW per chassis; 15 L/min of 30°C water with 10°C rise and orifice-balanced branches keeps worst-case DRAM junction ≤70°C against an 85°C throttle.

Latency (bounded, not average): lxy_repeater compare ≈0.1 ns; HFR hop ≈1–2 ns; worst-case on-board route (8×8) 14 hops ≈14–28 ns; spine traversal (7 layer hops) ≈7–14 ns; link serialization for a 4 KB message at the 16 GB/s fabric link ≈256 ns (the dominant term); doorbell wake ≈1 ns. The end-to-end activation bound for a 4 KB cross-node message is therefore ≈0.3 μs, dominated by link serialization, and it scales linearly with message length (≈64 ns per KB). The closed form verified in Section 4.3 is the sum of these terms: hop count × per-hop delay plus message bytes divided by the link rate; the co-simulation checks the hop-count and serialization terms exactly and treats contention as a separate, compile-time-scheduled term.

3.5 Co-simulation verification results

Table 3 summarizes the co-simulation runs on the 48-node chassis (3×4×4, deterministic seed, three parallel fabric slices). Every scenario passes all five oracle checks of Section 2.15. Figure 5 plots the per-scenario distributions.

sweep's mean latency of 25 cycles matches the closed form of Section 3.4 exactly, so per-hop accounting is exact rather than approximate. load delivers all 450 routed flits byte-exact despite sinks that accept only one byte in four; its 558-cycle worst case is a backpressure artifact of a slow consumer, not a routing anomaly. hotspot's 8,279-cycle worst case on its corner node is exactly the serialization and DMA-throttled drain of kilobyte-class tokens behind the 1-in-8 gate—and not a single byte is dropped. stress refuses all 36 corrupt-CRC flits—the count matching the software twin's refusals and the stub's corrupt_out verdicts exactly—while the remaining 1,051 activations fire kernels whose results are byte-exact against the golden model. replay's two runs produce 2.1 MB of bit-identical delivery logs across three slices, machine-checking deterministic replay. vcsweep's routed flits arrive within the same 24–27 cycle window despite concurrent pass-through traffic across all four virtual-channel classes, confirming the arbitrated egress tree adds no measurable delay at this load.

The 512-node chassis (8×8×8, eight parallel slices) exercises the full spine at scale under all six scenarios of Section 2.15: sweep at 24/27/31 cycles (min/mean/max), vcsweep likewise at 24/27/31, load with 18,782 wire bytes at 18/157/583 cycles, hotspot at 25/2,878/8,311 cycles, stress refusing 79 corrupt messages with 2,773 byte-exact activations at 12/632/8,287

Table 3: Co-simulation verification results (48-node chassis, 3×4×4, seed 1, three parallel slices).

Scenario	Routed	Pass-through	Wire bytes	Activations	Corrupt refused	Latency min/mean/max (cycles)
sweep	48	0	1,104	48	0	24/25/27
vcsweep	48	24	1,656	48	0	24/25/27
load	450	50	18,281	450	0	9/149/558
hotspot	380	20	151,191	380	0	27/2,888/8,279
stress	1,087	65	164,914	1,051	36	12/671/8,279
replay	289	11	52,132	285	4	14/758/8,255

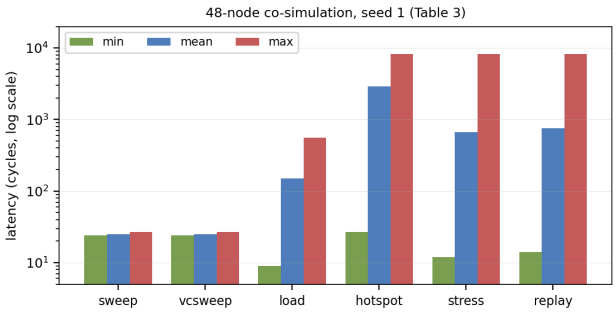


Figure 5: Per-scenario latency distributions from the 48-node co-simulation (Table 3, seed 1). Sweep hits the closed form exactly; the hotspot and stress tails are sink backpressure, and no scenario drops a byte.

cycles, and replay reproducing all eight slices' delivery logs bit-identically across two runs (2,053,924 bytes). The compiler path adds a two-expert program—a byte-sum kernel with silicon bias +7 and a 16-weight dot kernel with bias +3—on the same chassis: five dispatched tokens produce five doorbell activations, five byte-exact kernel results, zero refusals, and 14–26 cycle latencies.

The reasoning behind these checks is what makes them sound. Byte conservation is only testable because routing is single-path and deterministic: on a fabric with multiple legal routes the oracle would need a routing model of its own, whereas here the injected byte count is the delivered byte count by construction, and any divergence is a gate-level bug. The doorbell accounting is sound because the hardware verdict and the software twin are independently implemented—the HDL doorbell and CRC datapath on one side, the Go twin on the other—so their agreement is evidence, not tautology. Replay is sound because the fabric has no hidden state: if any queue, counter, or configuration bit persisted between runs, the second delivery log would diverge. And the sweep latency check is exact because contention-free schedules are compile-time properties (Section 2.8): the simulator runs the same schedule the AOT compiler emits, so the closed form is checked against the gates, not against a second software model. The same argument covers the arbitrated reverse path: the DES of Section 2.15 shares no code with the Verilog merge tree yet reproduces its delivery stream byte-for-byte on a mixed four-class program and matches the same closed form on an idle chassis, so the egress arbitration is pinned down by two independent implementations.

The host-to-chassis path adds four self-checking testbenches outside the co-simulation harness, each pinning down one interface boundary of Sections 2.16–2.17. tb_host_bmc.v drives the SPI console through bridge and arbiter into the shared register file and observes a correctly framed flit leave the spine ingress, boot-done assert, and the dispatch counter read back over the same serial link. tb_dma_lpddr6.v moves a four-word payload host→LPDDR6→host through the extended register window: the module's activity counters report exactly four writes and four reads, array contents match at the word-mapped address, and the readback is byte-exact across 27 absorbed refresh stalls. tb_sodimm_lpddr.v repeats the identical sequence against DDR4- and DDR5-class SODIMM timing profiles (CAS waits of 15 and 36 cycles against LPDDR6's 4, refresh intervals tightened to 780 cycles) and passes all three generations unchanged—the protocol stack carries no timing assumption of any single DRAM generation. tb_pcb_link.v verifies the interconnect models of Section 2.18: exact multi-stage latency, a 200-packet/1,408-byte stream delivered byte-exact in order, and triple-link activity counters matching injection counts.

Beyond these transport proofs, the same harness executes the full numeric forward pass of a real transformer. It runs the actual Gemma-4 26B-A4B checkpoint (26.5 B parameters, BF16) entirely in software—real weights, the real 262,144-token byte-fallback BPE tokenizer, real multi-position attention, and a real top-8-of-128 mixture-of-experts router—with no torch or transformers runtime. Every activation tensor is rounded to BF16 between operations, matching the reference storage regime, including the router's normalized expert weights. On the greedy-decode prompts tested the engine is token-for-token identical to a Hugging Face bf16 reference through the terminating turn token (the reference harness itself is not shipped; the repository test asserts the same token ids): the checkpoint's own weights answer "What is the capital of Germany" with "The capital of Germany is Berlin." and stop, and both the Germany and France chat prompts reproduce the reference's first eight tokens exactly. Those routing decisions run against the same gates the fabric proof exercises, with the router's top-8 weights landing bit-for-bit on the reference's BF16 grid. As a hardware cross-check, a real 16×16 tile of a layer-0 router weight matrix and a real embedding activation are loaded into the BF16 systolic MAC array of Section 2.9 and evaluated under gate-level simulation

Table 4: DUV MAC ASIC specifications (reference node; first-order estimates).

Parameter	Specification
Process	14 nm DUV (28 nm option), mature-node, high-yield; ≈60 mm^2 die—comfortably inside the DUV reticle and the mature-node yield sweet spot
Clocks	1 GHz fabric/MAC clock; 10.7 Gb/s/pin LPDDR6 I/O via on-die PLL; source-synchronous fabric forwarding
MAC array	192 FP64 FMA lanes; 384 FP16 lanes (2× packing); 768 INT8 lanes (4× packing)
Peak throughput	≈384 GFLOPS FP64; ≈768 GFLOPS FP16; ≈1.5 TOPS INT8
Memory interface	dual 192-bit LPDDR6 controller (384-bit aggregate), 4×48-bit channel groups each, 10.7 Gb/s/pin, ≈512 GB/s; bus SECDED plus on-die ECC pass-through (Sections 2.10 and 2.16)
Fabric interface	AXI-Stream slave (RX) and master (TX), 128-bit @ 1 GHz, ≈16 GB/s per direction; wormhole framing, end-to-end CRC per message
Doorbell DMA	byte counter, running CRC, DEST compare; DOORBELL_TRIG/DOORBELL_ACK and NODE_ERR on the sideband (Section 2.9)
On-chip SRAM	≈1–2 MB: elastic input buffer of at least one maximum-length message (compiler-set capacity), MAC staging, CRC/DMA workspace
Power	≈10 W total (≈9 W MAC array, ≈1 W fabric I/O) at 0.8 V core, 1.2 V I/O, 1.1 V LPDDR6 PHY
Package	Variant A: socketed LGA/card-edge, ≈800–850 pins, ≈20 mm from the two CAMM2 sockets; Variant B: mounted on a module substrate

(`iverilog/vvp`), reproducing an exact BF16-FMA software reference on every output—linking real model weights to the arithmetic units the architecture specifies, independent of the software forward pass. Beyond the isolated tile, the full integrated node datapath of Section 2.20 is machine-checked on the same real weights: a selectable node MAC runs a complete expert pass, a 16-wide slice of the checkpoint's layer-0 weights runs a real decoder block (attention, KV, dense MLP, top-2 MoE) that reproduces the corresponding BF16 reference word for word, and the gating ASIC's top-1 decision dispatched as one fabric flit lands on the node and matches the software BF16 twin—so the router, the fabric, and the compute ASIC are verified as one composed pipeline rather than three separate pieces.

3.6 The DUV MAC ASIC: detailed specifications

Table 4 is the complete datasheet outline for the reference node's ASIC (Variant A of Section 2.4). Every figure is a first-order estimate consistent with the node budget of Table 2.

The pin groups of Table 4 map onto the socket as follows: LPDDR6 PHY ≈290 pins (192 DQ, 24 DQS pairs, CA/CS/CK); fabric ≈270 pins (128-bit data in each direction plus valid/sop/eop/ready and a forwarded clock); sideband 4 pins; clock and test (reference oscillator, JTAG, configuration straps) ≈20; the remaining ≈250 pins are power and ground—the majority of any high-speed package. All high-speed groups are length-matched within the package substrate, and the PHY-to-socket escape is a controlled-impedance bundle consistent with the board outline of Section 2.4.

Two sizing remarks complete the picture. First, the MAC array is the die's minority: at 14 nm, 192 FP64 FMA lanes plus the 4× INT8 packing occupy roughly a third of the ≈60 mm^2, the LPDDR6 PHY and its escape another third, and SRAM, fabric I/O, and clocking the remainder—so the silicon is memory-interface-bound, exactly as the roofline of Section 3.2 predicts, and there is ample DUV margin to grow the array without moving the ridge point. Second, the ASIC holds no instruction path and no operating-system interface; its only program is the routing-table entry, resident kernel weights, and the KERNEL_CONST bias constant loaded through the POST sequence of Section 2.5, which is what keeps the pinout static across every workload the chassis can run.

The central router bridge is a second, distinct die: a CPU-less bridge that owns the chassis management plane (Section 2.17). Unlike the compute nodes, which are fixed-function MAC ASICs, the bridge is programmable—but by the host, not by itself: it accepts the host's routing tables, weight blobs, and dispatch commands over a host link (Section 4.2), performs the topology discovery and gating that the host's firmware drives it through, and injects the resulting flits into the fabric. The host runs the bridge's firmware, so the same silicon serves a Raspberry Pi, a microcontroller, or a workstation without change. The bridge die is smaller than the compute die (no MAC array, no 192-bit LPDDR6 PHY, no processor, no boot ROM, and no timer), uses the same mature-node process, carries a 64 KB staging SRAM, and is socketed and cold-swappable for serviceability.

The MAC array is not a monolithic unit but a collection of specialized compute units, each optimized for a different precision and operation class. The reference node instantiates twelve representative unit types, selected at compile time by the kernel mapper (Section 2.7); the RTL folds every FMA/ALU/MAC/select-add datapath into one CU-selected ‘arith_unit‘ (CU 0–11), and the 16-bit FP systolic array into one CU-parameterized ‘fp_mac_array‘, so the operation class is a build/compile choice rather than a second RTL family. The node exposes all of them through a single ‘CU_TYPE‘ selector (Section 2.20):

The ‘fp32_alu‘ unit deserves particular attention: it provides the division operation that the FMA units lack, and division is essential for normalization layers (layer norm, batch norm, softmax denominator). The ALU implements restoring binary long division with a 28-cycle latency, alongside 3-cycle MIN/MAX/CMP and 4-cycle FMA-path ADD/SUB/MUL. A dedicated ‘fp32_alu_chip‘ wrapper integrates the ALU with the PNM fabric's AXI-Stream interface: it deserializes 4-byte FP32 operands from the flit byte stream, feeds them to the ALU, and serializes the result back, all while running the CRC half of the doorbell discipline in hardware. This chip is the compute unit that sits behind the PE tile stub (Section 2.9) for layer norm and activation workloads.

Table 5: Compute unit taxonomy (per-node; reference configuration).

Unit	Precision	Pipeline latency	Operation class	Workload mapping
fp16_fma	FP16	3 cycles	Fused Multiply-Add	FP16 transformer inference, mixed-precision stencil halos
bf16_fma	BF16	3 cycles	Fused Multiply-Add	MoE expert weight-streaming, dense MLP projections (default)
fp32_fma	FP32	3 cycles	Fused Multiply-Add	High-precision layer norms, attention softmax denominators
fp64_fma	FP64	3 cycles	Fused Multiply-Add	FP64 stencil computation, scientific HPC kernels
fp16_mac_array	FP16	variable	Systolic MAC array	FP16 attention QKV projections
bf16_mac_array	BF16	variable	Systolic MAC array	BF16 attention QKV projections (default)
fp32_alu	FP32	3–28 cycles	ALU (ADD, SUB, MUL, DIV, MIN, MAX, CMP)	Layer norm, softmax, activation functions
int8_mac	INT8	2 cycles	Multiply-Accumulate	Quantized inference, INT8 weight-stationary kernels
int4_mac_array	INT4, INT8	variable	Systolic MAC array	INT4 quantized inference (4× density vs BF16)
fp4_mac_array	FP4 (E2M1)	variable	Systolic MAC array	FP4 quantized inference (4× density vs BF16)
mxfp4_mac_array	MXFP4	variable	Systolic MAC array	MXFP4 block-scaled (OCP microscaling) inference
ternary_acc	-1, 0, +1 × INT8	3 cycles	Select-add accumulator	BitNet b1.58 weight streaming (1.58 bits/weight)

The quantized precision spectrum extends the taxonomy down to 4-bit floating point. The ‘fp4_mac_array‘ and ‘mxfp4_mac_array‘ units operate on FP4 (E2M1) values packed two per byte, dequantizing each nibble to fixed point and accumulating to INT32—4× the packing density of BF16, equal to that of INT4, at the cost of reduced input precision. Following the OCP Microscaling convention, MXFP4 additionally carries one shared 8-bit block scale per 32-element block, so a run of weights can recover the exponent range that a bare 4-bit field cannot represent. A 4-bit value is too narrow to multiply directly, so both units follow the same dequantize-then-multiply convention as the INT4 path: the systolic array accumulates fixed-point products in INT32 and the wider accumulator absorbs the reduced input precision, with the block scale folded into the mantissa shift. The ‘weight_dequant‘ unit was extended to the same four modes (INT4, INT8, FP4, MXFP4), and the co-simulation ‘QuantMode‘ offers ‘int4‘, ‘fp4‘, and ‘mxfp4‘ weight-only options alongside the default BF16. These narrower formats are a deployment option for bandwidth-bound weight streaming, not the default: BF16 remains the reference precision, and the closed-form latency bounds of Section 3.4 hold for every unit type because the array pipeline depth, not the data width, sets the critical path.

The spectrum extends one step below 4-bit, to BitNet-style 1.58-bit weights[42]. A ternary weight is restricted to -1, 0, +1, so multiplying by it is not a multiplication at all but a signed conditional add/subtract: the ‘ternary_acc‘ unit computes one such accumulate step—a ternary weight against an INT8 activation, accumulated into signed INT32—on the same two-stage pipeline as the INT8 MAC, and it carries no multiplier (a select-add accumulator rather than a MAC), and the node of Section 2.20 selects it as ‘CU_TYPE‘ 8. The per-tensor (or per-channel) scale $s_x \cdot s_w$ is applied after the integer accumulate, host-side, exactly as the existing per-expert scale is. Ternary weights are roughly 1.58 bits per parameter against BF16’s 16—a $\sim$10× reduction in both resident footprint and weight-streaming traffic—which sharpens the capacity-bound argument of Section 4.5 without moving the fabric-bound ceiling, since that ceiling is set by the pipeline depth and the wire format, not the data width. The unit is validated bit-exact against a select-add oracle in gate-level simulation (node ‘CU_TYPE‘ 8), and the assembly builder ships it as a node product (‘implementations/boards/node_ternary.json‘).

All FMA modules share an identical interface: ‘clk, rst_n, a, b, c, valid_in‘ producing ‘result, valid_out‘ with a 3-cycle pipeline. The systolic MAC arrays have a variable latency depending on the matrix dimensions. The PE tile carries a ‘CU_TYPE‘ selector—bias-add, BF16 FMA, INT4/FP4/MXFP4 MAC, or the transport stub’s legacy ‘USE_FMA‘ bias-add mode—so the same interface hosts any unit behind it (Section 2.20). The model compiler (Section 2.7) assigns unit types based on the operation: MoE experts use BF16 FMA (weight-stationary), attention projections use BF16 systolic arrays, layer norms use FP32 ALU (for division), and dense MLP layers use BF16 FMA. This assignment is static and immutable: the same node always runs the same unit type for the same workload, which is what makes the deterministic latency bounds of Section 3.4 possible.

3.7 Configuration spectrum: SODIMM deskside units to spatial supercomputers

The architecture is a family of machines, not a single SKU, and the degrees of freedom are deliberately orthogonal. Four axes vary independently without touching the others: the memory module generation on each node, the compute-unit taxonomy instantiated behind the PE tile, the router-chip tier managing the chassis, and the physical extent (boards stacked into layers). Every axis is exercised by running RTL or tooling in the repository, and every combination speaks the same wire format (Section 2.9), honors the same doorbell discipline,

and compiles from the same .pnm program schema—which is the concrete meaning of modularity here.

Memory axis: CAMM2 versus SODIMM. The flagship node pairs the MAC ASIC with two LPCAMM2/LPDDR6 modules[16, 17] for the ≈512 GB/s of Table 2, but the identical ASIC, repeater, doorbell, and firmware run against commodity SODIMM modules[30] where a deployment calls for desktop economics instead of density. A dual-channel DDR5-5600 SODIMM provides ≈45 GB/s per module with capacities to 96 GB today; a dual-slot node reaches ≈90 GB/s and 192 GB. Node bandwidth drops roughly 3×, but Section 3.2's roofline already places both flagship workloads below even the reduced ridge points: weight-streaming at 1–2 FLOP/byte remains bandwidth-bound on DDR5 exactly as on LPDDR6, under the same static schedule and the same closed form. The claim is machine-checked rather than asserted: tb_sodimm_lpddr.v drives one unmodified protocol stack against DDR4-, DDR5-, and LPDDR6-class timing parameters and verifies byte-exact DMA in all three generations—what changes between them is only the module's wait-state count and refresh cadence, precisely the parameters the behavioral module exposes.

Scaling down: GPU-class deskside units. The spine is optional hardware below chassis scale. A single 4×4 board—16 nodes—forms a deskside unit in the multi-TB class (up to 2 TB at CAMM2 densities; 1.5 TB on 96 GB SODIMMs) drawing under 250 W: the power envelope of one workstation, the capacity of a small rack, and no spine stack at all. With every destination on the sender's own layer there is nothing above the board for the vertical fabric to do: the latency closed form loses its spine term (zero effective spine hops), virtual-channel classes 1 and 2 go uninstantiated, and the spine may be omitted entirely—the board's egress merge trees terminate directly in the router instead of an lxy_repeater column, or retain that one column as a passive turn-around stub if a later stack-up is anticipated. Nothing else changes: source-routed matching compares coordinate bytes against local parameters, never absolute topology position, so the wire format, doorbell discipline, and .pnm program schema are byte-for-byte those of Section 2.9. The MCU-tier router variant (Section 2.17) suffices at this scale—16 routing entries fit in 4 KB of SRAM—and a single-board computer console reaches the PNM window over the SPI bridge of Section 2.17 without PCIe. Workloads whose resident set fits a few hundred GB—MoE expert pools, FP64 stencil subdomains, seismic image stacks—serve interactively from this unit; the compile-model/run-driver pipeline targets it by passing board dimensions, with no source change. At this end of the spectrum the machine competes not with datacenter HBM but with the local-GPU status quo quantified in Section 3.3: a workstation GPU holds at most 80–96 GB behind its 300 W package, while the deskside unit holds an order of magnitude more capacity for comparable power.

Scaling out: spatial supercomputers. Chassis-level scaling replicates the reference configuration and joins replicas through a second routing level: each chassis keeps its deterministic internal closed form, and cross-chassis traffic leaves through the spine root as ordinary flits toward a chassis-interconnect tier—the extension point Section 4.4 identifies. Because routing tables are immutable per boot session and schedules are compile-time artifacts, a multi-chassis job is assembled by concatenating per-chassis schemas at batch boundaries; no runtime coherence protocol appears anywhere in the stack. The result is a spatial supercomputer whose building block is a ≈$1M, 10 kW board stack rather than an interposer-bound accelerator tray, so scale-out economics inherit the commodity-memory advantage rather than diluting it.

Discussion

4.1 Failure model and recovery

Losses are observable before they become failures. Every injected message has a compile-time latency budget, and its source expects DOORBELL_ACK within that window (Section 2.4). A missing ACK—whether the message was dropped, truncated, or refused at the doorbell—raises NODE_ERR for that destination; there is no silent hang. NODE_ERR assertion triggers a four-step protocol: (1) quarantine the node from the live topology schema; (2) discard in-flight flits destined for it (the affected set is exactly enumerable on a single-path fabric); (3) recompute—the source re-injects the work toward a healthy replica or the reinitialized node, under the same doorbell discipline; this is functional recomputation, not bit-exact replay of the failed node's memory (Section 2.10); (4) re-export the amended schema at the next batch boundary. Routing tables are immutable within a boot session, so step (4) is a controller-coordinated reload, not runtime adaptation: the router regenerates the AOT tables against the amended schema and loads them during the batch boundary, when the fabric is quiet. The chassis continues in degraded mode with no restart. A failed spine lane is handled at the physical layer—periodic link re-training excludes the dead lane and the remaining lanes re-lane transparently—so routing tables need no change for lane degradation; total spine failure partitions the chassis (the Conclusion section).

4.2 Host interface and I/O

The host attaches to the central router bridge over a byte-level host link—SPI on a Raspberry Pi, or the same protocol from a microcontroller—and a workstation host reaches the same window over PCIe. The host streams initial weights at boot, injects input tokens, and drains outputs. Runtime token traffic is kilobyte-class: the host injects one input token per generated token, and the 270 fabric dispatches per token (Section 2.6) are generated from the resident expert map, so the uplink carries token-rate traffic—roughly 4×10^6 tokens/s

of kilobyte-class tokens, about 4–30 GB/s—not dispatch-rate traffic. The weight-upload bound of Section 2.5 is set by this uplink, and a link of B bytes per second uploads 64 TB in $64 \times 10^{12}/B$ seconds. The fabric's internal dispatch rate (Section 2.1) exceeds the uplink rate precisely because the per-token dispatch multiplier is realized inside the fabric. The host never touches the compute fabric directly.

4.3 Verification and formal properties

Deadlock freedom (sketch). Every flit is assigned one of four virtual-channel classes in strictly increasing order: (0) on-board egress (X-then-Y dimension order), (1) spine ascent, (2) spine descent, (3) on-board delivery (X-then-Y from spine attachment to destination). Upward and downward spine flows are separate virtual channels (classes 1 and 2), which is why the count is four, not three. Intra-class dependencies are acyclic by standard results; cross-class dependencies only point from lower to higher classes. The global channel dependency graph is therefore acyclic, and by Dally and Seitz the fabric is deadlock-free[13, 14].

The buffering that virtual channels require lives only at the two kinds of state points in the design: the lxy_repeater spine attachments (per-class elastic buffers) and the node DMA ports (Section 2.9). HFRs remain stateless re-timers: they carry no per-VC state and never decide a route, which is what keeps the "no queues on the data path" claim true for the forwarding elements. The per-VC attachment buffers are credited in the flow-control sense below, so a blocked flit occupies its class's buffer and cannot block another class.

Livelock freedom follows from minimal single-path routes. Consumption is enforced structurally: the per-VC elastic buffers at spine attachments and node ports hold at least one maximum-length message, and sources may not inject without destination buffer credit, so no class ever exhausts and blocks a lower class. The egress merge tree adds no deadlock surface of its own: its dependency graph is a DAG—board egress toward spine ascent, never across spine descent—so the packet-atomic round-robin merges of Section 2.15 cannot deadlock and the argument extends to the arbitrated reverse path. Worst-case latency for any coordinate pair is a closed-form expression—hop count $\times$ per-hop delay plus message length divided by link rate (Section 3.4)—and contention is absent by construction because the AOT scheduler (Section 2.8) assigns message schedules that respect every buffer's credit budget; the co-simulation verifies the hop-count and serialization terms exactly and treats the scheduler's credit feasibility as a compile-time property. Given identical inputs onto an identical fabric, every computation and message schedule reproduces bit-exactly (Section 2.10).

4.4 Limitations and future work

LPDDR6 (JESD209-6)[17] CAMM2 parts and high-capacity modules are forward-looking; the current JEDEC socketable module class, LPCAMM2 (JESD318), is LPDDR5X and trims per-node bandwidth to ≈136–153 GB/s with 48–64 GB modules today[16, 17]. The architecture accommodates both module classes with no change to the MAC ASIC or routing fabric. Manifold balancing requires CFD and hardware validation. Multi-chassis scale-out needs a second routing level and one additional virtual-channel class; the physical interconnect itself—a verified protocol-transparent optical link pair with asynchronous elastic buffering (Section 2.19)—is modeled in the repository, leaving the second-level router specification and the VC-field widening as the remaining design work. General lazy evaluation needs a dynamic heap this design omits. A dual-spine variant eliminates single-chassis partition mode. The liveness claims of Section 4.3 remain an argument—a simulation can exercise the fabric but cannot prove acyclicity of its channel-dependency graph[9]—while the transport claims (lossless delivery, byte-exact reproduction, doorbell refusal accounting) are machine-checked by the co-simulation of Sections 2.15 and 3.5.

Hotspots, errors, and scaling ceilings. The closed-form latency bounds of Section 3.4 are exact on an idle chassis; under hotspots, wormhole head-of-line blocking behind a backpressured sink can produce tree saturation whose queueing delay the closed form does not bound. The co-simulation's hotspot scenario verifies losslessness, not a latency bound under that contention; the mitigation is compile-time: hot-expert replication, layer-affine placement, and credit-feasible schedules (Section 2.12). Error handling is likewise end-to-end: CRC is checked only at the destination doorbell (Section 2.9), so a mid-fabric corruption is detected on the ACK timeout of Section 4.1 and costs a full-flit source retransmit; there is no per-hop retry and no fabric-level timeout below the doorbell's compile-time latency budget. The wire format and routing-bitmap fields bound the chassis: the 16-bit LEN field caps payloads at 64 KB (weight uploads span many flits), the routing bitmap's 4-bit Layer ID field (Section 2.1) caps a chassis at 15 routable layers, and the MODULE_ID nibbles cap a board at 16×16 nodes; the reference configuration uses 8 layers and an 8×8 board, and multi-chassis scale-out routes through a second level (Section 3.7) rather than widening these fields. The sideband control net is single-wire per signalling function per node, but the router-side aggregation of up to 512 wires per signalling function (DOORBELL_ACK, NODE_ERR, TOPOLOGY_RDY) is a pin and trace cost on the spine mezzanine and router package that the per-node four-pin sideband budget (Section 2.4) does not capture; the design accepts it in exchange for the single-wire control plane.

Fixed-function brittleness and resource limits. The fixed-function MAC array has no instruction decoder: a new operator or quantization format is a silicon respin, and the compiler, wire format, CRC coverage, and routing-bitmap layout are

co-locked and must change together. Model-class changes are absorbed by the compiler's re-derivation of placements, bitmaps, and dispatch plans from any config/safetensors pair (Section 3.1.1); instruction-set changes are not. Adaptation is batch-boundary-only: hot-expert replication and failure-schema re-export require a quiet-fabric routing-table reload (Section 4.1), so traffic shifts within a batch cannot be absorbed. The reference design is single-tenant: there is no isolation, multi-tenancy, or memory protection between concurrent workloads, and the AOT schedule assumes exclusive chassis ownership. KV capacity per attention node is bounded by the fixed 512-byte entry frames and 1,024 physical entries per directional bank (4,096 entries at the reference depth); long contexts beyond the 16K logical-entry window spill through the NVMe offload path at storage-class bandwidth. Weight-upload time is bounded by the host uplink, not the fabric: 64 TB through the dual PCIe Gen5 x16 uplinks takes approximately 12–15 minutes at a realistic sustained fraction of line rate (Section 2.5), because the upload is a long sequential write burst the fabric simply forwards.

Validation scope. This paper's empirical claims rest on a validated behavioral substrate, not on fabricated silicon. The Verilog fabric, compute units, routers, and memory models are exercised by self-checking testbenches and the Go co-simulation at 100 MHz with one byte per cycle (Section 2.3); there is no place-and-route, timing closure, or silicon measurement behind the bandwidth, power, thermal, or cost figures. The only synthesis evidence is a generic Yosys smoke test (the companion repository's synth.sh): every routing gate, the doorbell, a BF16 FMA unit, and the host bridge map to technology-independent cells (23 to 3,519 cells for the routing and compute blocks, at logic depths between 1 and 32 levels), which shows that the RTL elaborates and synthesizes but uses no standard-cell library and backs no timing, area, power, or bandwidth figure. The 1 GHz 128-bit links of Section 2.3 are never clocked in simulation—the $\approx$160$\times$ gap is stated explicitly there—and the nanosecond latencies of Section 3.4 are projections of the closed form at the characterized link rate. The DRAM, NVMe, and PHY blocks are behavioral stubs with parameterized CAS latency and refresh scheduling: they verify control flow, framing, and byte-enable behavior, not DRAM silicon. The LPDDR6 controller and PHY integration on a mature 14/28 nm node is the riskiest silicon claim in the design; the stubs validate control flow and framing, but no analog PHY exists in the RTL. The MoE gating dispatch exercised by the co-simulation is a deterministic token-seeded hash, not a trained gating network (disclosed in Section 2.12); a learned 128-expert BF16 router needs roughly 0.7 MB of projector weights, which is the weight store the shipped gating ASIC provides, so only the gating evaluation latency is a projection. The real-checkpoint forward pass of Section 3.5 is a software result—it validates the numeric engine against a BF16 reference, not the RTL—and a full trained forward pass on the fabric remains future work; the gate-level evidence for real weights is limited to the router-weight tile of Section 3.5. CRC-16/CCITT-FALSE is a probabilistic integrity check, not a proof: at the characterized fabric rate of $\approx 2.5 \times 10^8$ 8 KB frames per second, the per-frame residual is 1 in 2^{16}—approximately one undetected corruption every 50 days at a BER of 10^{-15}, and one every few seconds at 10^{-12}—so bit-exactness claims apply to the simulated traffic and to the residual rate stated here. The 512-node scale evidence is a routed wire record of 18 KB plus 2.0 MB across two replay runs; the co-simulation partitions the chassis into parallel slices, it does not instantiate 512 nodes of application state. Power figures are first-order design-point estimates consistent with the event-accounting checks of the repository's power monitor; the DRAM-read energy implied by full-bandwidth traffic is a projection to be validated, not a measurement, and thermal figures await the CFD validation of this section. BOM figures exclude mezzanine connectors, chassis assembly, and the isothermal manifold. What is machine-checked is exactly this: byte-exact delivery and replay, doorbell refusal accounting, and closed-form latency equality on the 48- and 512-node chassis; a real-checkpoint numeric forward pass that is greedy-token-identical to a BF16 reference on the tested prompts; and a bit-exact integrated compute datapath—the gating ASIC's decision dispatched as one fabric flit, the node MAC running that expert, a full decoder block against a BF16 reference, and the chain of all three across model layers (Sections 2.15, 2.20, and 3.5). The paper is therefore a position argument carried by a verified behavioral substrate, and its quantitative claims should be read as projections on top of that substrate.

Reliability claims and their validation scope. The only integrity mechanism the co-simulation actually exercises is the end-to-end CRC-16/CCITT-FALSE check of Sections 2.9 and 2.10: the stress scenario injects corrupt-CRC flits and the doorbell refuses every one, with the hardware verdict matching the software twin's refusal count exactly (Section 3.5). LPDDR6 on-die ECC, the MAC ASIC's bus SECDED, and the background scrub pass of Sections 2.10 and 2.16 are design claims, not validated artifacts: the repository's DRAM and PHY models are behavioral stubs with no ECC datapath, no SECDED encoder or decoder exists in the RTL, and no testbench or co-simulation run exercises them—the scrub's mean-time-to-undetected-mis-correction bound is a sized projection, not a measured property. No functional-safety certification (IEC 61508 SIL, ISO 26262 ASIL, or equivalent) is claimed for the fabric or the router chip. The probabilistic residual of the frame CRC—1 in 2^{16} per frame at the rate stated in the previous paragraph—bounds every integrity claim in this section, and it applies unchanged to both memory baselines. A DDR5 baseline changes the ECC comparison but not its validation scope: commodity DDR5 RDIMMs provide conventional bus ECC with sideband check bits alongside the on-die refresh ECC that DDR5 carries internally; LPDDR6 relies on on-die ECC; and in both cases the controller SECDED of Section 2.10 is the same unvalidated design claim.

DDR5 baseline economics. The SODIMM deployment of Section 3.7 trades bandwidth for cost. A dual-slot DDR5 node carries ≈192 GB at ≈$2/GB 2025–26 spot pricing (roughly $400 of DRAM) against ≈$1,900 for the 128 GB LPDDR6 CAMM2 of Table 2—a ≈7× lower memory cost per byte, partially offset by doubled module count and socket area at equal capacity. Node power stays inside the ≈15 W envelope: the DDR5 dual-channel pair draws roughly 5–8 W against ≈5 W for the LPDDR6 CAMM2 module. The bandwidth-bound conclusion of Sections 3.2 and 3.7 is unchanged, but the DDR5 node's ≈3× lower bandwidth raises the per-GB/s cost of the fabric and spine provisioned behind it, making the DDR5 baseline a capacity-maximizing rather than bandwidth-maximizing point on the spectrum of Section 3.7. Multi-chassis scale-out compounds the difference: the second routing level this section identifies is dimensioned from per-node bandwidth, so a DDR5 baseline carries ≈3× less cross-chassis bandwidth per chassis, and every published chassis figure (Table 2, Sections 3.5 and 4.4) is stated for the LPDDR6 reference only—redeployment on DDR5 requires re-planning schedules, not re-architecting them, because the compile-time scheduler of Section 2.8 absorbs the new bandwidth ratios unchanged. The 2025–26 DRAM spot prices behind both baselines are shortage-driven and cyclical (Section 3.3); the structural comparison—commodity modules and mature-node silicon against interposer-bound HBM—does not depend on them.

The central router chip is a single point of failure, and this paper treats that fact explicitly rather than designing around it. Chassis availability is bounded by the router's availability multiplied by the fabric's: the router is socketed and cold-swappable, POST re-discovers the chassis in under a second (Section 2.5), and all in-flight traffic drains through the stateless fabric while the spare boots, so a router replacement costs a sub-second dispatch outage. There is no live router failover—dual-router handover of the JIT tables is future work—and the reference design accepts the resulting availability bound in exchange for a single programmable silicon. An HFR, lxy_repeater, or node failure degrades the machine by exactly that node (Section 4.1); a router failure halts dispatch until the spare is in place.

4.5 Impacts: capacity-bound workloads at commodity economics

The workload landscape has bifurcated. On one side, training and interactive inference remain bandwidth-dominated, served by GPU accelerators with HBM. On the other side, a growing class of capacity-bound workloads—Mixture-of-Experts transformer inference, scientific HPC stencil computations, climate modeling, seismic imaging, and large-scale numerical simulation—are bottlenecked not by peak FLOPs but by the memory capacity required to hold resident data[21, 22, 23, 26]. These workloads share a common memory profile: the arithmetic intensity is low (0.3–2 FLOP/byte), the data sets are large (terabytes to petabytes), and the dominant cost is moving data from memory to compute, not the compute itself[5]. NVIDIA already reported roughly 40% of datacenter revenue from inference in 2024—'probably understated'—and its subsequent 'inference inflection' commentary is consistent with the broader shift toward capacity-bound serving[2]. Capacity-bound serving is weight-stationary: for every token the serving hardware streams the model's weights once from memory—decode arithmetic intensity is ≈2 FLOP/byte at batch 1 in FP8, far below the ridge point of any current GPU—and performs only a couple of operations per byte read[24, 26]. Serving throughput is bounded by memory bandwidth, and what can be served is bounded by memory capacity: fitting weights plus KV cache into HBM, not peak FLOPs, decides the batch a server can hold, which is why industry analyses call HBM capacity 'throughput capital'[26]. MoE serving in particular is memory-capacity-bound: the binding resource is KV cache plus activated expert weights, not the token count[27]. Stencil workloads are even more memory-bound: a 7-point FP64 stencil has an arithmetic intensity of 0.3–0.5 FLOP/byte, far below the ridge point of any current accelerator[5].

This is precisely the regime this architecture is built for. The reference chassis is a weight-stationary serving machine: every parameter and KV cache entry resides in commodity LPDDR6 pools that bypass the interposer and lithography bottlenecks constraining HBM, the router's closed-form coordinate function dispatches tokens to resident experts (Sections 2.8 and 2.12), the doorbell makes each activation a sub-microsecond event (Sections 2.9 and 3.4), and the spine sizing rule (Section 2.1) sustains 10^8–10^9 fabric dispatches per second—at 270 dispatches per generated token (Section 2.6), a token rate of roughly 4×10^6 tokens/s at the favorable placement of Section 2.12, within an order of magnitude of the $\approx 10^7$ tokens/s of today's largest production deployments[24]. For HPC stencil workloads, the rigid X/Y grid maps directly to the motherboard topology: each grid subdomain becomes a physical node, halo exchanges are single-hop dimension-order routes, and the deterministic latency bounds enable replacing MPI with compile-time coordinate functions. A GPU serving rack uses HBM for exactly the same job—holding weights and KV cache—but at 300 W per package with capacity capped by interposer reticle limits, versus approximately 15 W per node with capacity bounded only by the number of commodity modules the chassis can hold. The roofline argument of Section 3.2 is sharpened, not contradicted, by the serving regime: the served workload is even more memory-bound than the training-style analysis assumes.

Chassis capacity is also a function of weight encoding, and here the precision spectrum of Section 3.6 compounds with the capacity argument. The same 64 TB of commodity DRAM holds roughly 32 trillion BF16 parameters but about 324 trillion ternary (BitNet b1.58) weights—a factor of ten—for models whose quality is preserved by 1.58-bit quantization[42]. Capacity-bound serving and low-precision

weight formats are thus multiplicative rather than additive levers: the cost per served parameter falls both because commodity DRAM is cheap and because each byte carries four ternary weights instead of half a BF16 weight. The 'pnmhost benchmark' cost table reports this full weight-format spectrum ($\sim$32 T BF16 / 64 T INT8 / 128 T INT4 / 324 T ternary parameters per 64 TB chassis), and the ternary encoding is realised in silicon by the multiplier-free select-add accumulator of Section 3.6 (node 'CU_TYPE' 8, validated bit-exact).

The structural consequence is that HBM demand is coupled to exactly the workload class where this architecture substitutes commodity DRAM. Every inference token served on LPDDR6 displaces an HBM-resident weight read; because inference is the majority and fastest-growing share of AI compute[21, 22, 23], serving that share on capacity-optimized memory slows the industry's HBM demand growth. The trajectory is steep: inference rose from roughly a third of datacenter AI compute in 2023 to half in 2025[21], Gartner projects it overtaking training in 2026—55% of AI-optimized IaaS spending, rising beyond 65% by 2029[22]—and McKinsey has it reaching more than half of all AI compute by 2030[23]. Inference is also the workload class whose economics compound: every served token is an operating cost, token volume grows superlinearly as reasoning and agentic models multiply the work per task, and Google alone reported processing $\approx 3\times10^{15}$ tokens per month in 2026[24]. HBM's disproportionate wafer footprint—the proximate cause of the 2025–26 DRAM shortage and its price spike[25]—then frees fabrication capacity back toward commodity DRAM as the mix shifts. Training retains HBM's bandwidth economics; inference—the majority—moves to capacity economics, where commodity modules with no interposer dependency and mature-node yields above 90% dominate on both physical and economic grounds. The memory market bifurcates, and this machine is built for the inference half.

4.6 Design-for-test and manufacturing readiness

Design-for-test follows the same concentration principle. Both dies' clock-gating cells carry a scan-mode override (the latch-based gate passes the test clock unconditionally when scan_en asserts, so gated domains remain observable during scan shift), and every asynchronous reset crossing passes through a two-flop synchronizer (rst_sync.v) so reset-domain crossings are scan-compatible rather than metastability hazards. The PHY blocks self-test through loopback with no external partner, and the optical links' CRC-on-arrival check (Section 2.19) doubles as an in-field signal-integrity monitor. Manufacturing hand-off artifacts ship alongside the RTL rather than after it: SDC timing constraints for the 100 MHz reference domain with false paths enumerated for the combinational ROM/SRAM reads and async resets (pnm_soc.sdc), a device tree source describing the full router-chip memory map for NOMMU Linux boot (pnm_soc.dts), and the assembly schema of the companion repository that composes verified modules into board-level netlists. What remains vendor-dependent—SerDes analog characterization, package bond maps, floorplan extraction—is explicitly outside the RTL's claims, consistent with this paper's practice of labeling engineering estimates rather than claiming them away.

Conclusion

This paper has specified a distributed spatial Processing-Near-Memory architecture that bypasses the HBM capacity wall through three physical properties that no market correction can alter: commodity LPDDR6 CAMM2 modules with no interposer dependency[16, 17], mature-node DUV logic with high yields and established supply chains, and deterministic wormhole routing that eliminates runtime scheduling overhead[13, 14]. By treating the physical motherboard as an immutable dataflow graph and exporting its topography as a bespoke ISA, the system eliminates cache coherency, operating-system overhead, and the thermal density of advanced interposer stacks. Routing is coordinate arithmetic on a single-spine tree with dimension-order on-board paths; deadlock freedom follows from four monotonically ordered virtual-channel classes with credited per-class buffers at the two state points in the design; the hardened doorbell makes activation atomic, idle-free, and loss-observable through a single ACK wire; and the isothermal parallel manifold sustains thermal viability with enforced flow balance. A CPU-less router bridge at the spine root provides the management plane on a single socketed, cold-swappable die; the host—a Raspberry Pi, a microcontroller, or a workstation—runs its firmware, so the same silicon serves every deployment.

The reference chassis—64 TB, 262 TB/s aggregate node-local bandwidth, approximately 197 TFLOPS FP64, approximately 15 W per node—demonstrates that the physical constraints of interposer packaging, EUV yield, and thermal density favour distributed commodity memory by a wide margin. At current DRAM prices, the cost advantage is approximately 24$\times$ per byte against HBM, but the structural advantage—no interposer, no EUV, no 300 W thermal hotspots—persists regardless of market cycles.

The same fabric specification scales down to SODIMM-based deskside units—DDR4 or DDR5 SODIMM modules with relaxed CAS latency—and scales out to multi-chassis spatial supercomputers via a second routing level, without changing the wire protocol, the doorbell discipline, or the compiler's program schema (Section 3.7). Target workloads—including MoE transformer inference, FP64 stencil computation for scientific HPC, climate and seismic imaging codes, and functional data-based evaluation—sit in the bandwidth-bound regime on the precision they actually run, where mature-node silicon is already sufficient[3, 4, 5], and they inherit deterministic fabric replay, compile-time latency bounds, and an honest availability model with a single swappable router as the only point of programmable state.

The full repository, including Verilog HDL sources, build scripts, and this manuscript, is available at https://github.com/im-BowenGu/PNM-Arch. The hardware designs (the Verilog HDL sources) are released under the CERN Open Hardware Licence Version 2 Strongly Reciprocal (CERN-OHL-S)[20], the build and simulation code under AGPL-3.0-or-later, and this manuscript under CC BY-SA 4.0.

Acknowledgments

The author thanks their research mentor for guidance throughout this project, the anonymous referees for their constructive feedback, and the open-source community for developing the tools and ethos that made this work possible.

References

[1] W. A. Wulf, S. A. McKee. Hitting the memory wall: implications of the obvious. *ACM SIGARCH Computer Architecture News*. Vol. 23, pg. 20-24, 1995.

[2] J. Choquette. Nvidia hopper h100 gpu: scaling performance. *IEEE Micro*. Vol. 43, pg. 9-17, 2023.

[3] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, J. Dean. Outrageously large neural networks: the sparsely-gated mixture-of-experts layer. *arXiv preprint* arXiv:1701.06538, 2017.

[4] W. Fedus, B. Zoph, N. Shazeer. Switch transformers: scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*. Vol. 23, pg. 1-39, 2022.

[5] S. Williams, A. Waterman, D. Patterson. Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*. Vol. 52, pg. 65-76, 2009.

[6] J. Backus. Can programming be liberated from the von Neumann style? a functional style and its algebra of programs. *Communications of the ACM*. Vol. 21, pg. 613-641, 1978.

[7] M. D. McIlroy, E. N. Pinson, B. A. Tague. Unix time-sharing system: forward. *Bell System Technical Journal*. Vol. 57, pg. 1899-1904, 1978.

[8] J. Liedtke. On micro-kernel construction. Proceedings of the fifteenth ACM symposium on Operating systems principles. pg. 237-250, 1995.

[9] G. Klein, K. Elphinstone, G. Heiser, J. Andronick, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish, T. Sewell, H. Tuch, S. Winwood. seL4: formal verification of an OS kernel. Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles. pg. 207-220, 2009.

[10] N. P. Jouppi et al. In-datacenter performance analysis of a tensor processing unit. Proceedings of the 44th annual international symposium on computer architecture. pg. 1-12, 2017.

[11] J. B. Dennis. First version of a data flow procedure language. Programming Symposium. pg. 362-376, 1974.

[12] I. Barron, P. Cavill, D. May, P. Wilson. Transputer does 5 or more MIPS even when not used in parallel. *Electronics*. Vol. 56, pg. 109-115, 1983.

[13] W. J. Dally, C. L. Seitz. Deadlock-free message routing in multiprocessor interconnection networks. *IEEE Transactions on Computers*. Vol. C-36, pg. 547-553, 1987.

[14] W. J. Dally, C. L. Seitz. The torus routing chip. *Distributed Computing*. Vol. 1, pg. 187-196, 1986.

[15] JEDEC Solid State Technology Association. High bandwidth memory (HBM) DRAM standard. JESD235D, 2021.

[16] JEDEC Solid State Technology Association. Compression attached memory module (CAMM2) common standard. JESD318, 2023.

[17] JEDEC Solid State Technology Association. Low power double data rate 6 (LPDDR6) SDRAM standard. JESD209-6, 2025.

[18] C. Lattner et al. MLIR: scaling compiler infrastructure for domain specific computation. 2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO). pg. 2-14, 2021.

[19] Message Passing Interface Forum. MPI: a message-passing interface standard, version 4.0. 2021.

[20] CERN. CERN open hardware licence version 2 - strongly reciprocal (CERN-OHL-S). 2020.

[21] Deloitte. Why AI's next phase will likely demand more computational power, not less. *Deloitte Insights, TMT Predictions 2026*. 2025.

[22] Gartner. Gartner says AI-optimized IaaS is poised to become the next growth engine for AI infrastructure. Press release, October 15, 2025.

[23] McKinsey & Company. The cost of compute: a $7 trillion race to scale data centers. 2025.

[24] R. Raghuram, S. Wang. How to win the largest market in AI. *a16z*. 2026.

[25] TrendForce. Memory spot price update: DDR5 prices up 307% since September as module costs poised to surge. November 19, 2025.

[26] H. Menon. Inference curves: bandwidth, not compute, is the primary serving knob. *SemiAnalysis InferenceX*. 2026.

[27] Y. Yu, et al. Efficient MoE serving in the memory-bound regime: balance activated experts, not tokens. *arXiv preprint* arXiv:2512.09277, 2025.

[28] B. Gu. Breaking the HBM wall: Verilog HDL sources, Go co-simulation harness, and manuscript repository. https://github.com/im-BowenGu/PNM-Arch, 2026.

[29] Linux Foundation. Linux kernel RISC-V architecture documentation: NOMMU (no-MMU) support. https://www.kernel.org/doc/html/latest/arch/riscv/, 2025.

[30] JEDEC Solid State Technology Association. Double data rate 5 (DDR5) SDRAM standard. JESD79-5C, 2024.

[31] F. Devaux. The true processing in memory accelerator. *2019 IEEE Hot Chips 31 Symposium (HCS)*. pg. 1-24, 2019.

[32] Y.-C. Kwon, S. H. Lee, J. Lee, et al. A 20nm 6GB function-in-memory DRAM, based on HBM2 with a 1.2TFLOPS programmable computing unit using bank-level parallelism, for machine learning applications. *2021 IEEE International Solid-State Circuits Conference (ISSCC)*. pg. 350-352, 2021.

[33] J. H. Kim, S. Kang, S. Lee, et al. Aquabolt-XL: Samsung HBM2-PIM with in-memory processing for ML accelerators and beyond. *2021 IEEE Hot Chips 33 Symposium (HCS)*. pg. 1-26, 2021.

[34] J. H. Kim, et al. Aquabolt-XL HBM2-PIM, LPDDR5-PIM with in-memory processing, and AXDIMM with acceleration buffer. *IEEE Micro*. Vol. 42, pg. 20-30, 2022.

[35] S. Ghose, A. Boroumand, J. S. Kim, J. Gomez-Luna, O. Mutlu. Processing-in-memory: A workload-driven perspective. *IBM Journal of Research and Development*. Vol. 63, pg. 3:1-3:19, 2019.

[36] J. Ahn, S. Yoo, O. Mutlu, K. Choi. PIM-enabled instructions: A low-overhead, locality-aware processing-in-memory architecture. *Proceedings of the 42nd Annual International Symposium on Computer Architecture (ISCA '15)*. pg. 336-348, 2015.

[37] J. Gomez-Luna, I. E. Hajj, I. Fernandez, C. Giannoula, G. F. Oliveira, O. Mutlu. Benchmarking a new paradigm: Experimental analysis and characterization of a real processing-in-memory system. *IEEE Access*. Vol. 10, pg. 52565-52608, 2022.

[38] T. Mann. Why did Nvidia really drop $20B on Groq? *The Register*. December 31, 2025.

[39] Cerebras Systems. CS-3: the WSE-3 wafer-scale engine. Vendor product documentation. 2025.

[40] T. P. Morgan. Taalas etches AI models onto transistors to rocket-boost inference. *The Next Platform*. February 19, 2026.

[41] T. Mann. AMD acquires AI chip startup Taalas to boost inference performance by etching models into silicon. *The Register*. August 6, 2026.

[42] S. Ma, H. Wang, L. Ma, L. Wang, W. Wang, S. Huang, L. Dong, R. Wang, J. Xue, F. Wei. The era of 1-bit LLMs: all large language models are in 1.58 bits. *arXiv preprint* arXiv:2402.17764, 2024.